AKTU

B.Tech III-Year

5th Semester

CS IT & CS Allied

GATEWAY CLASSES

# DBMS: Database Management System

# ONE SHOT Revision

# Unit-1

# DBMS: Database Management System

## AKTU Unit-1 Syllabus

Introduction: Overview, Database System vs File System, Database System Concept and Architecture, Data Model Schema and Instances, Data Independence and Database Language and Interfaces, Data Definitions Language, DML, Overall Database Structure. Data Modeling Using the Entity Relationship Model: ER Model Concepts, Notation for ER Diagram, Mapping Constraints, Keys, Concepts of Super Key, Candidate Key, Primary Key, Generalization, Aggregation, Reduction of an ER Diagrams to Tables, Extended ER Model, Relationship of Higher Degree.

| AKTU PYQs | | |
|---|---|---|
| Q.1 | What are the different types of models in DBMS? Explain them | AKTU 2022-23 /2020-21/2018-19/ 2017-18 |
| Q.2 | Explain different types of keys<br>OR Difference between primary key and unique key.<br>OR Explain super key, Primary key, candidate key. | AKTU 2020-21/2017-18/ 2021-22 |
| Q.3 | What is the significance of physical data independence | AKTU 2020-21 |
| Q.4 | List four function of database administrator(DBA) | AKTU2022-21 |
| Q.5 | Explain DML, DDL | AKTU 2017-18/18-19/20-21 |
| Q.6 | What is instance and schema of database | AKTU 2017-18/2022-23 |
| Q.7 | Difference between physical and logical independence | AKTU 2022-23 |
| Q.8 | What is data abstraction? Explain the level of data abstraction<br>OR How data abstraction is achieve in DBMS? | AKTU 2022-23/ 2020-21 |

| ...AKTU PYQs | | |
|---|---|---|
| Q.9 | Briefly discuss about the difference between data system and file system | AKTU 2020-21/ 2019-20 |
| Q.10 | What is data independence and its types | AKTU 2020-21/ 2021-22 |
| Q.11. | Explain the difference between physical level, conceptual level and view level | AKTU 2017-18 |
| Q.12 | What is the role of database administrator | AKTU 2017-18 |
| Q.13 | What is database language | AKTU 2018-19 |
| Q.14 | Distinguish between strong entity set and weak entity set.ER diagram to illustrate the weak entity set | AKTU 2022-23/ 2019-20 |
| Q.15 | Explain ER diagram in detail. | AKTU 2018-19 |
| Q.16 | What is the key of a relation? Explain the difference between primary key, super key and candidate key. | AKTU 2015-16 |

| ...AKTU PYQs | | |
|---|---|---|
| Q.17 | Give example of a simple, composite, attribute of an entity | AKTU 2015-16 |
| Q.18 | Difference between DBMS and RDBMS | AKTU 2018-19 |
| Q.19 | Explain briefly the overall structure of DBMS and also explain its components | AKTU 2018-19 |
| Q.20 | Describe the three schema architecture why we need mapping in schema level | AKTU 2022-23 |
| Q.21 | Explain all user of database in detail | AKTU 2021-22 |
| Q.22 | List some problem with file system | AKTU 2022-23 |
| Q.23 | What is DBMS ,data and meta data | AKTU 2022-23 |
| Q.24 | State the procedural DML and non-procedural DML | AKTU 2020-21 |
| Q.25 | What are the four advantages of file system over database | AKTU 2022-23 |

# Difference Between [AKTU 2022-23]

| Aspect | Data | Database | DBMS (Database Management System) |
|---|---|---|---|
| Definition | Raw facts and figures without context | A structured collection of related data | Software used to create, manage, and control access to the database |
| Purpose | To be processed into meaningful information | To organize and store data efficiently | To allow users and applications to interact with the database |
| Form | Numbers, words, images, symbols, etc. | Tables, records, files, etc. | Software programs (e.g., MySQL, Oracle, SQL Server) |
| Example | 101, Raja, A+, 5000 | A table storing student details | MySQL software managing that student database |
| Use in Real World | Collected through forms, sensors, surveys, etc. | Used in schools, hospitals, banks to store records | Used by IT admins, developers, analysts to work with data |

GW
GATEWAY CLASSES

# difference between-

| Aspect | Data | Database | DBMS (Database Management System) |
|---|---|---|---|
| Dependency | Base for creating databases | Depends on data | Depends on both data and database |
| User Interaction | Not directly usable by users | Can be accessed using DBMS | Provides interface to access and manipulate the database |
| Operations Possible | No operations | Stores organized data | Perform operations like insert, delete, update, and query |

GW
GATEWAY CLASSES

# DBMS Administrator [AKTU 2022-2 /2017-18]

A DBMS Administrator, also called a Database Administrator (DBA), is a person responsible for managing and maintaining a database system using DBMS software.

| Role | Explanation |
|---|---|
| 1. Database Installation | DBA installs the database software (like Oracle, MySQL, SQL Server) on servers and sets up the system for use. |
| 2. Configuration | Configures memory, storage, and settings according to the needs of the application or organization. |
| 3. Data Security | Ensures data is safe from unauthorized access by creating users, assigning roles, and setting permissions. |
| 4. Backup and Recovery | Schedules regular backups and restores data if there's a failure or accidental deletion. Protects against data loss. |
| 5. Performance Tuning | Monitors database performance and improves speed by optimizing queries, indexing, and system settings. |

GW
GATEWAY CLASSES
GW
GATEWAY CLASSES

## DBMS Administrator

| Role | Detailed Explanation |
|---|---|
| 6. Data Integrity | Ensures that the data entered in the database is accurate and consistent. Applies rules and constraints. |
| 7. User Management | Creates, manages, and removes database users. Controls what each user can see or modify. |
| 8. Monitoring and Troubleshooting | Constantly checks the database for errors, slowdowns, or unusual activity. Fixes issues quickly. |
| 9. Data Migration & Upgrades | Transfers data from one system to another during upgrades or when changing software, without losing any data. |
| 10. Documentation | Keeps records of all changes made to the database, like updates, backups, and user changes. Helpful during audits. |

# Types of DBA [AKTU 2017-18]

| Type of DBA | Role / Responsibility | Key Tasks |
|---|---|---|
| 1. System DBA | Focuses on installation, configuration, and server-level database tasks | Install DBMS, manage storage, configure OS/database, patch updates |
| 2. Application DBA | Supports specific applications that use the database | Work with developers, support application queries, procedures, schema |
| 3. Performance DBA | Specializes in improving speed and efficiency of the database | Query optimization, indexing, monitoring slow queries, performance tuning |
| 4. Backup & Recovery DBA | Ensures data safety through proper backup and recovery procedures | Schedule backups, restore data after crash, manage disaster recovery plans |
| 5. Security DBA | Manages data access control and security policies | Create roles, assign privileges, prevent unauthorized access |

# Types of DBA

| Type of DBA | Role / Responsibility | Key Tasks |
|---|---|---|
| 6. Development DBA | Works closely with developers to create and maintain database code | Write SQL queries, stored procedures, functions, triggers |
| 7. Data Warehouse DBA | Manages data warehouses used for large-scale analytics and reporting | ETL (Extract, Transform, Load), OLAP systems, handle big data queries |
| 8. Cloud DBA | Manages databases hosted on cloud platforms like AWS, Azure, Google Cloud | Handle cloud deployment, auto-scaling, backup on cloud, cost optimization |
| 9. Infrastructure DBA | Focuses on the hardware and network infrastructure that supports database operations | Monitor disk usage, CPU load, storage planning, hardware configuration |

# What is a Database User? [AKTU 2021-22]

A database user is any person, application, or system that interacts with a database to perform operations such as storing, retrieving, updating, or managing data

## TYPES OF USERS

| User type | Description | Example |
|---|---|---|
| 1. Naive Users (End Users) | They use applications to access the database without knowing how the database works. They interact through interfaces (e.g., forms, buttons). | A person withdrawing money from an ATM. |
| 2. Application Programmers | Developers who write code (in Java, Python, etc.) that interacts with the database using SQL commands through programs. | A developer creating a mobile banking app. |
| 3. Sophisticated Users | They are skilled users who write direct SQL queries to access and analyze data. They do not use applications—they interact with the DBMS directly. | A data analyst writing SQL reports. |

# TYPES OF USERS

| User type | Description | Example |
|---|---|---|
| 4. Specialized Users | Users who write special database applications using complex data (e.g., graphics, scientific data). These apps are not typical business applications. | A scientist storing DNA sequence data. |
| 5. Database Administrator (DBA) | Responsible for managing the entire database system. Tasks include user access control, backup, recovery, security, and performance tuning. | The admin managing your college database. |

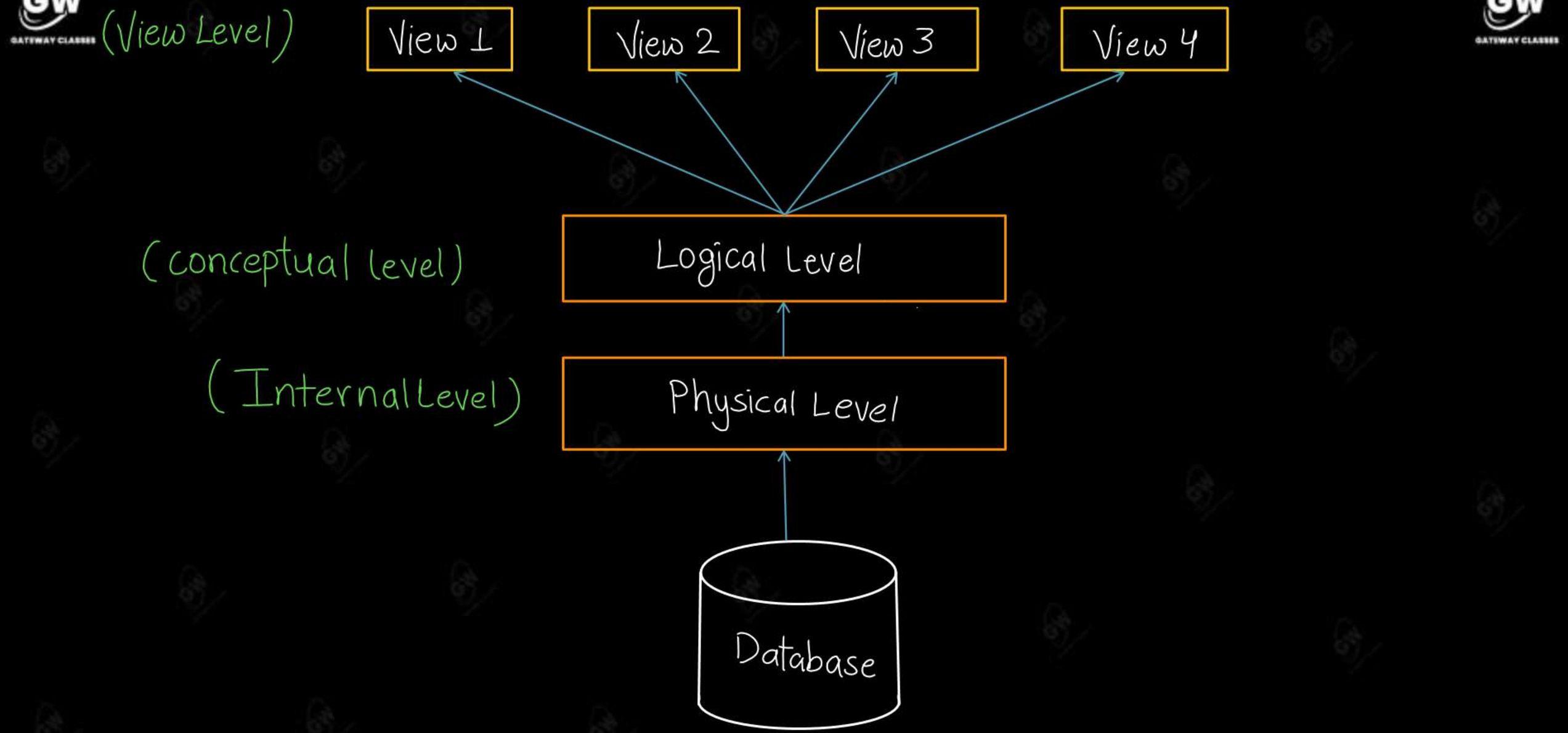
# Difference Between- [AKTU 2017-18]

| Point | File System | DBMS (Database Management System) |
|---|---|---|
| 1. Definition | A way to store and manage data in files. | A system to store, manage, and query data in databases. |
| 2. Data Access | Sequential access (mostly). | Can access data easily using queries (e.g., SQL). |
| 3. Redundancy | High data redundancy (duplicate data). | Reduces redundancy using normalization and relations. |
| 4. Data Integrity | Hard to maintain consistency. | Ensures data integrity using constraints. |
| 5. Security | Basic or no security. | Provides strong security with access controls. |
| 6. Backup & Recovery | Manual and difficult. | Automated and efficient backup and recovery. |
| 7. Concurrency | Poor support for multiple users. | Supports multiple users accessing data at the same time. |
| 8. Data Relations | Cannot easily link related files. | Stores related data using tables and keys. |
| 9. Query Language | No built-in query language. | Uses SQL to perform complex queries. |
| 10. Scalability & Efficiency | Less efficient for large data. | Designed for efficient and scalable data handling. |

# What is Database Abstraction [AKTU 2022-23/2020-21/ 2020-21]

Database abstraction means hiding the complex details of how data is stored, managed, and processed inside a database, while providing a simple interface to users.

| Level of abstraction | Description | Who Uses It | Example |
|---|---|---|---|
| 1. Physical Level | This is the lowest level. It defines how data is actually stored in memory or on disk – file structures, indexes, compression, etc. | Used by DBA/System Admin | - Data stored in .dat files or binary format.- Indexes (e.g., B-tree) are used to access data quickly. |
| 2. Logical Level | Describes what data is stored and how the tables are related. It defines tables, data types, relationships, and constraints. | Used by Database Designers | - STUDENT(roll_no, name, age)- COURSE(course_id, name)- Relationship: each student can enroll in many courses. |
| 3. View Level | Top-most level. It shows only selected parts of the data to users. Hides unwanted data to ensure simplicity and security. | Used by End Users | - A student logs in and sees only their own data.- A teacher sees marks of only their subject. |


(View Level)
View 1
View 2
View 3
View 4
(conceptual level)
Logical Level
(Internal Level)
Physical Level
Database

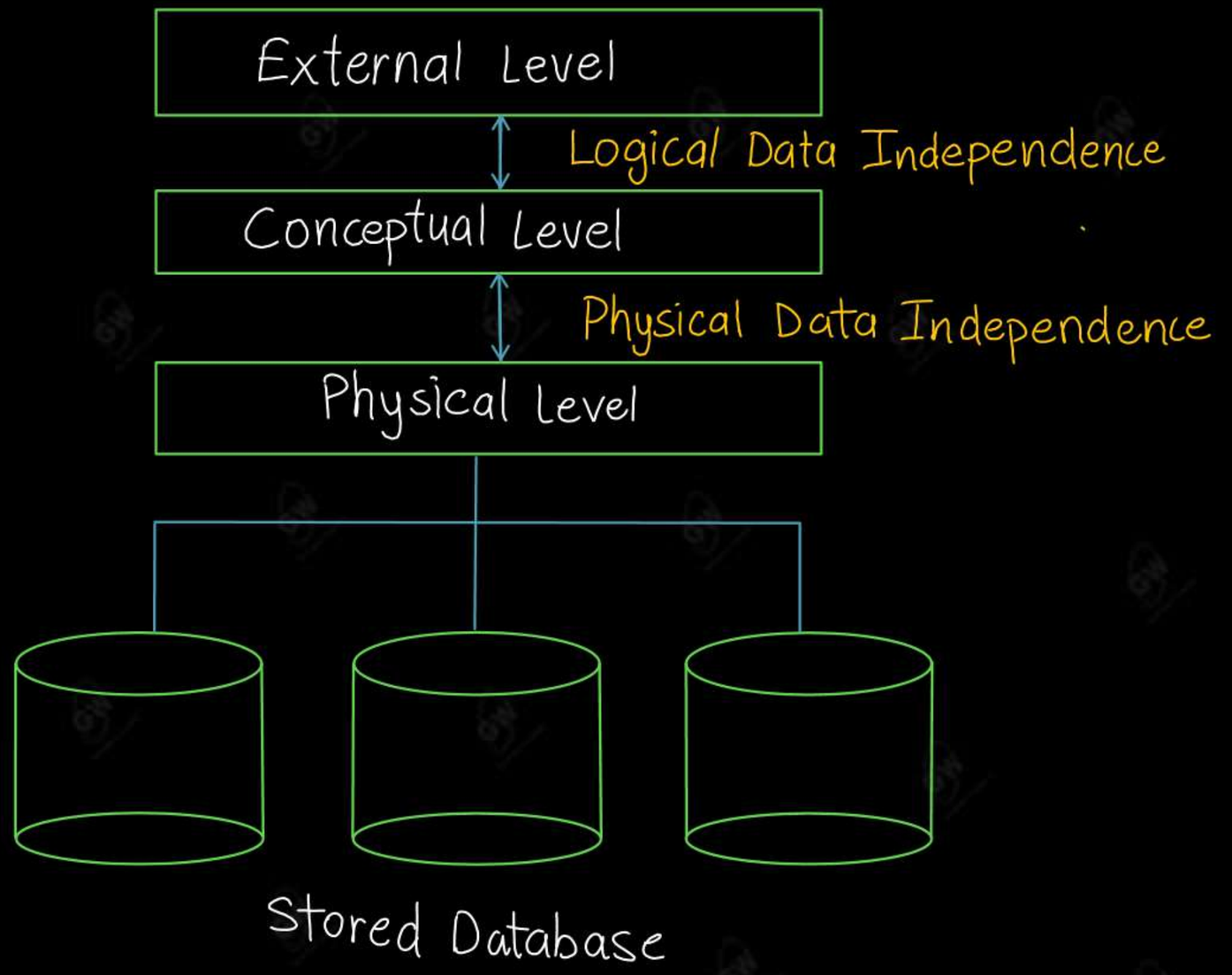
# What is Data Independence? [AKTU 2022-23/ 2019-20/ 2017-18]

Data independence means the ability to change the database structure at one level without affecting the higher levels.

## Types of data independence

| Type | Meaning | Example |
|---|---|---|
| 1. Logical Data Independence | Ability to change the logical schema (tables, attributes, relationships) without affecting external views or applications. | - Adding a new column in a table without changing the front-end form. |
| 2. Physical Data Independence | Ability to change the physical storage (file format, indexing) without changing the logical schema or programs. | - Moving from hard disk to SSD, or changing index type, but table definitions remain unchanged. |

# Data Independence in DBMS



# DBMS Architecture

DBMS architecture describes how different components of a database system interact — from user input to data storage. It determines how the database is designed, accessed, and maintained.

There are three main types of DBMS architecture:

## 1-Tier Architecture (Single Layer)

- In 1-tier architecture, the user directly interacts with the database.
- There's no separation between user interface and database logic.
- Everything (application, logic, and database) is handled on the same machine.

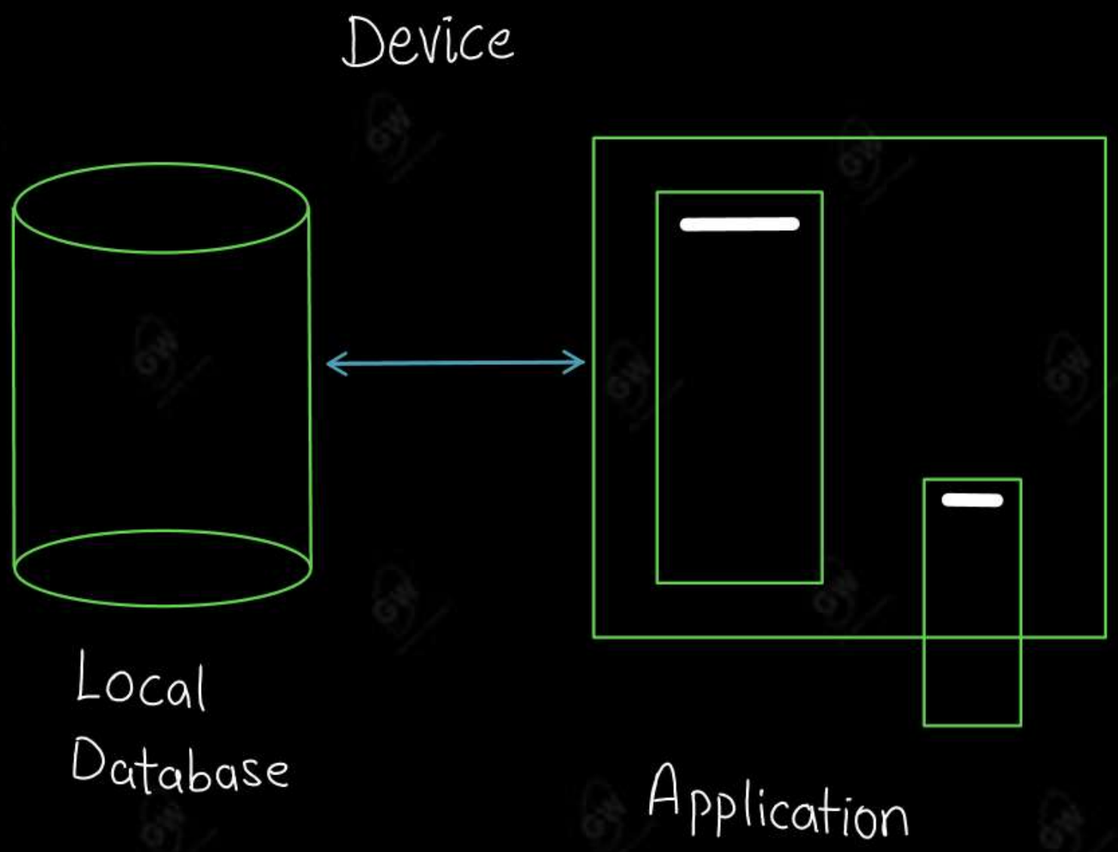
**Characteristics:**

- Simple and easy to set up.
- Used in local, small, or personal systems.
- Low security and scalability.

**Example:**

- MS Access: You open the file, view/edit data directly.
- SQLite in Python: Local DB embedded within the application

## 1-Tier Architecture (Single Layer)




## 2-Tier Architecture (Client-Server Model)

**Description:**

The application is divided into two layers:

Client (Front-end) – user interface/application.

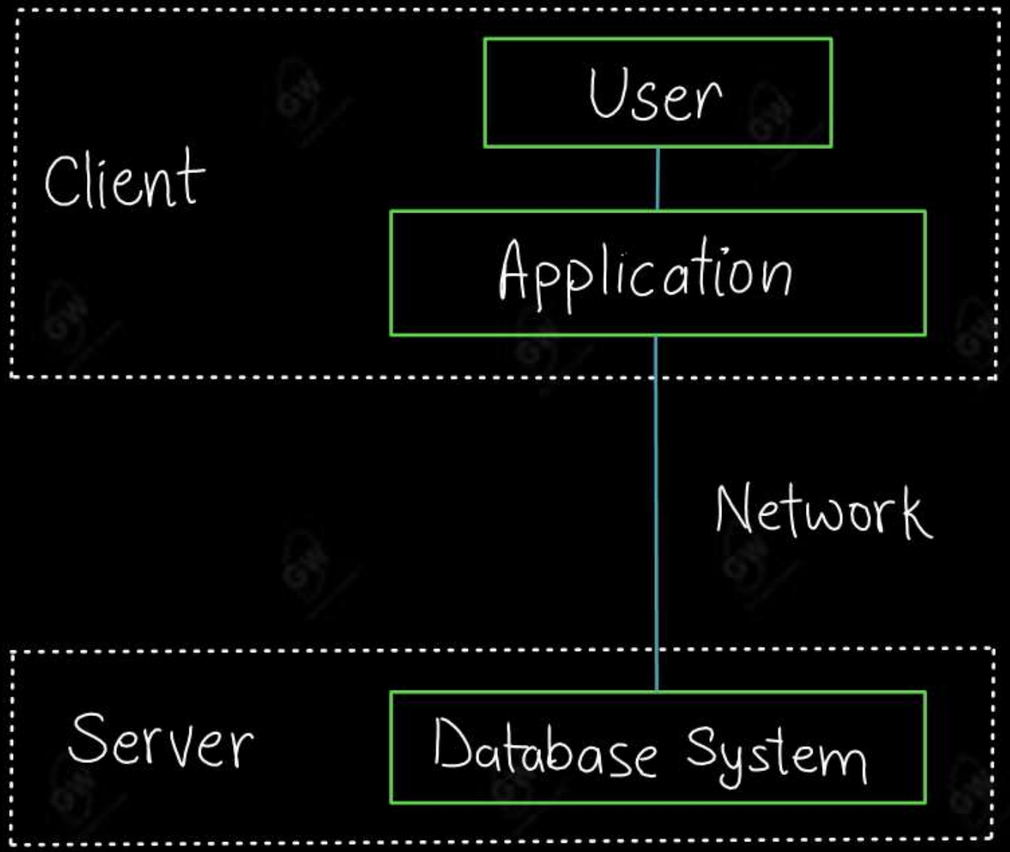
Database Server (Back-end) – where the database is stored and managed.

The client sends SQL queries to the server, and the server responds.

**Characteristics:**

- Better performance than 1-tier.
- Data is stored centrally.
- Suitable for small to medium systems.
- Security depends on client-server protocol

## 2-Tier Architecture




## 3-Tier Architecture (Web-Based/Enterprise Systems)

Divides the system into three layers:

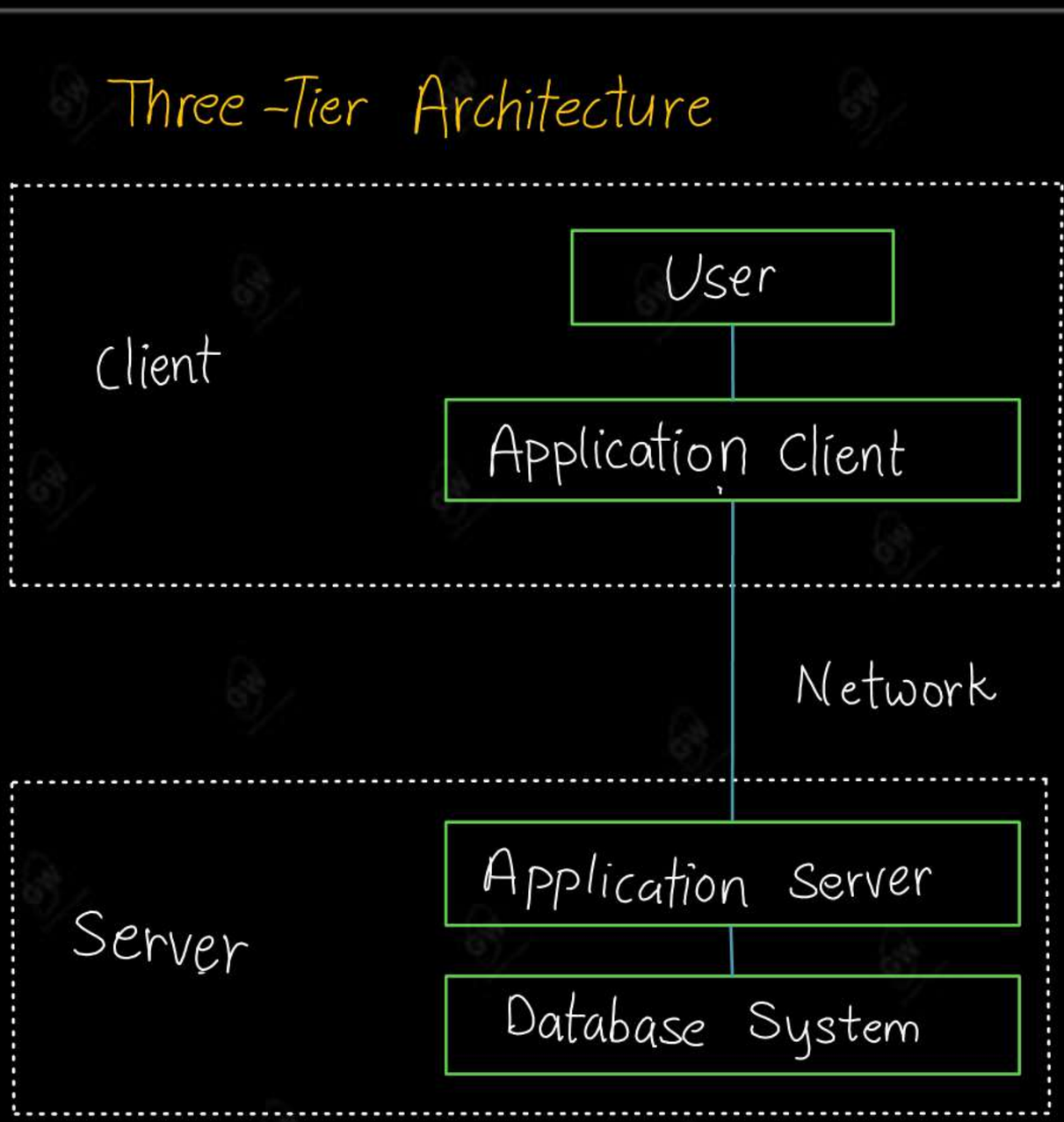
- Presentation Tier (Client) – user interface (e.g., browser, mobile app).
- Application Tier (Middleware/Server) – processes user requests, business logic.
- Database Tier – stores and manages data.

**Characteristics:**

- Highly secure, scalable, and maintainable.
- Suitable for large and complex systems.
- Each layer can be developed or updated independently.

## Example:

- A web browser sends login data → processed by server-side PHP/Java → fetches result from MySQL/PostgreSQL.
- Online banking, e-commerce platforms like Amazon



| Tier | Advantages | Disadvantages |
|---|---|---|
| 1-Tier | ➢ Simple to design and use<br>➢ Fast performance for single users<br>➢ No network needed | ➢ Not suitable for multiple users<br>➢ Very low security<br>➢ No data sharing |
| 2-Tier | ➢ Faster than 3-tier for small apps<br>➢ Direct communication with database<br>➢ Easy to maintain | ➢ Not scalable<br>➢ Client handles logic – high load<br>➢ Security is weaker than 3-tier |
| 3-Tier | ➢ Highly secure (clients can't access DB directly)<br>➢ Scalable and flexible<br>➢ Easier maintenance and updates | ➢ More complex design<br>➢ Slower than 2-tier for small apps<br>➢ Costlier to implement |

# Data model AKTU 2022-23 /2020-21/2018-19/ 2017-18

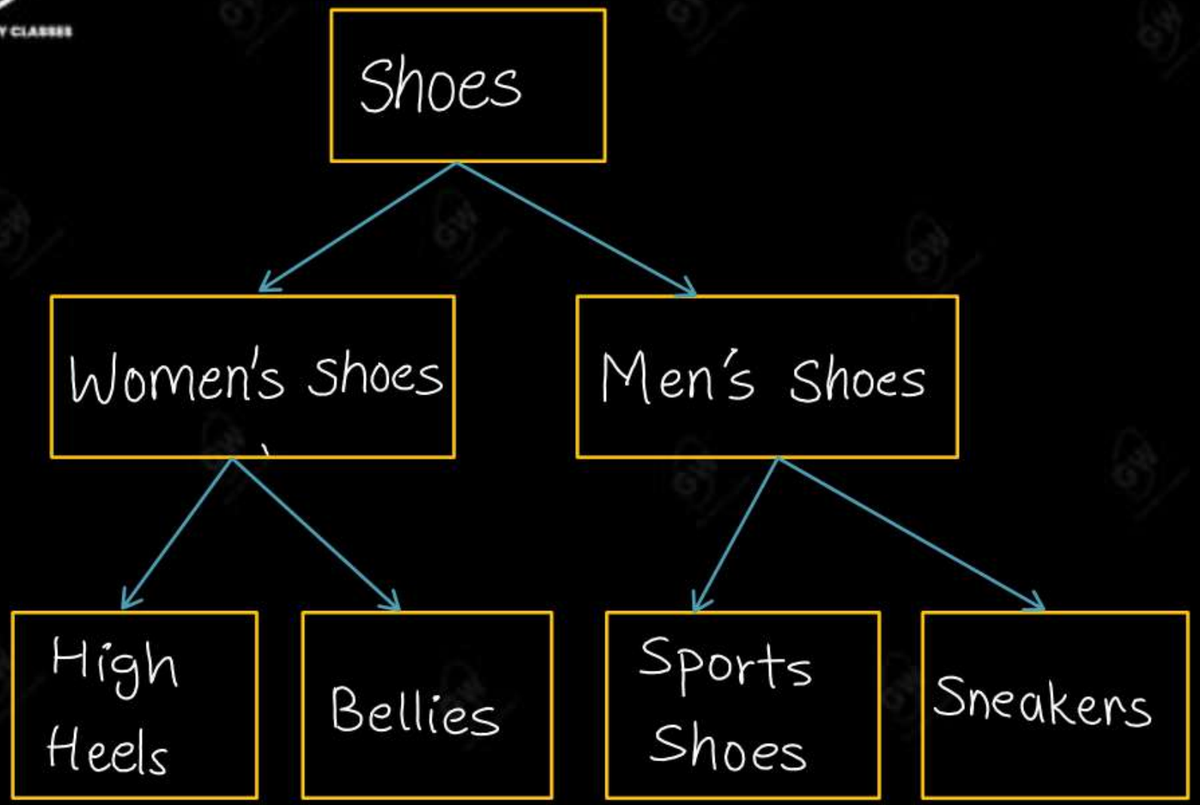
A data model is a conceptual framework that defines how data is structured, stored, related, and manipulated in a database.

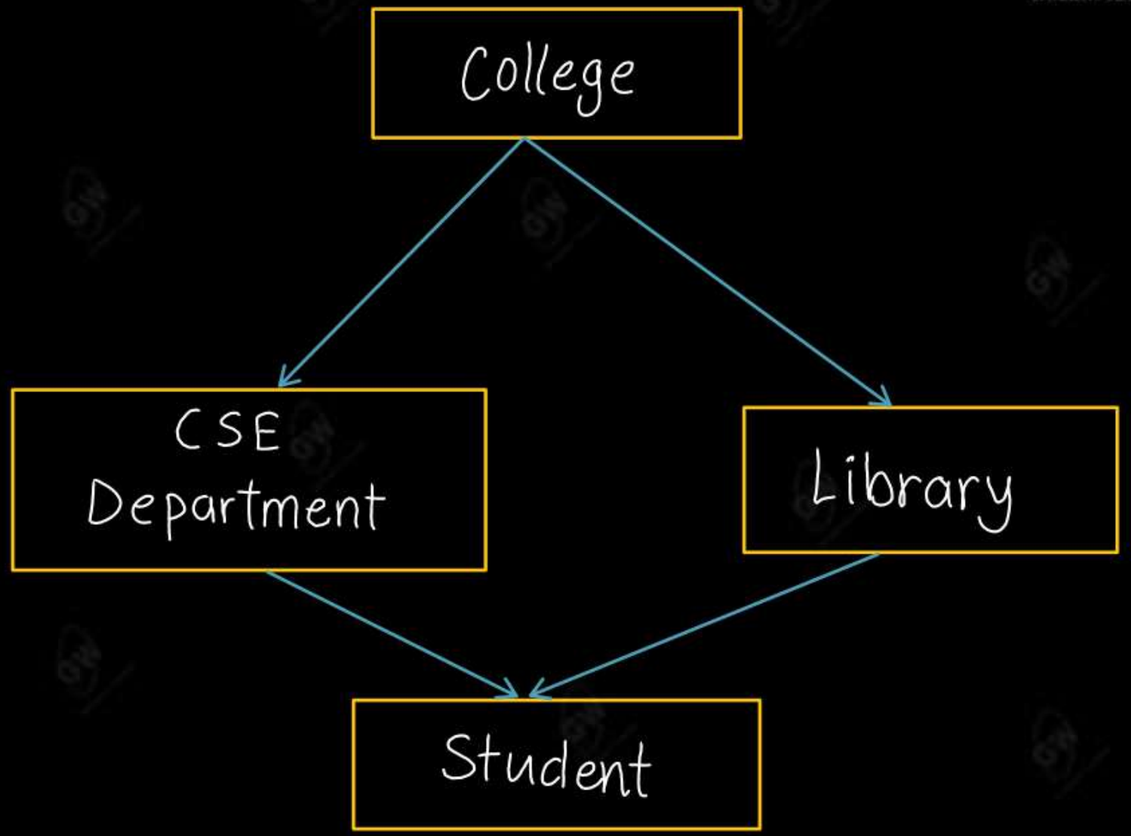
It provides a way to organize data, define relationships, and apply rules for data consistency and integrity.

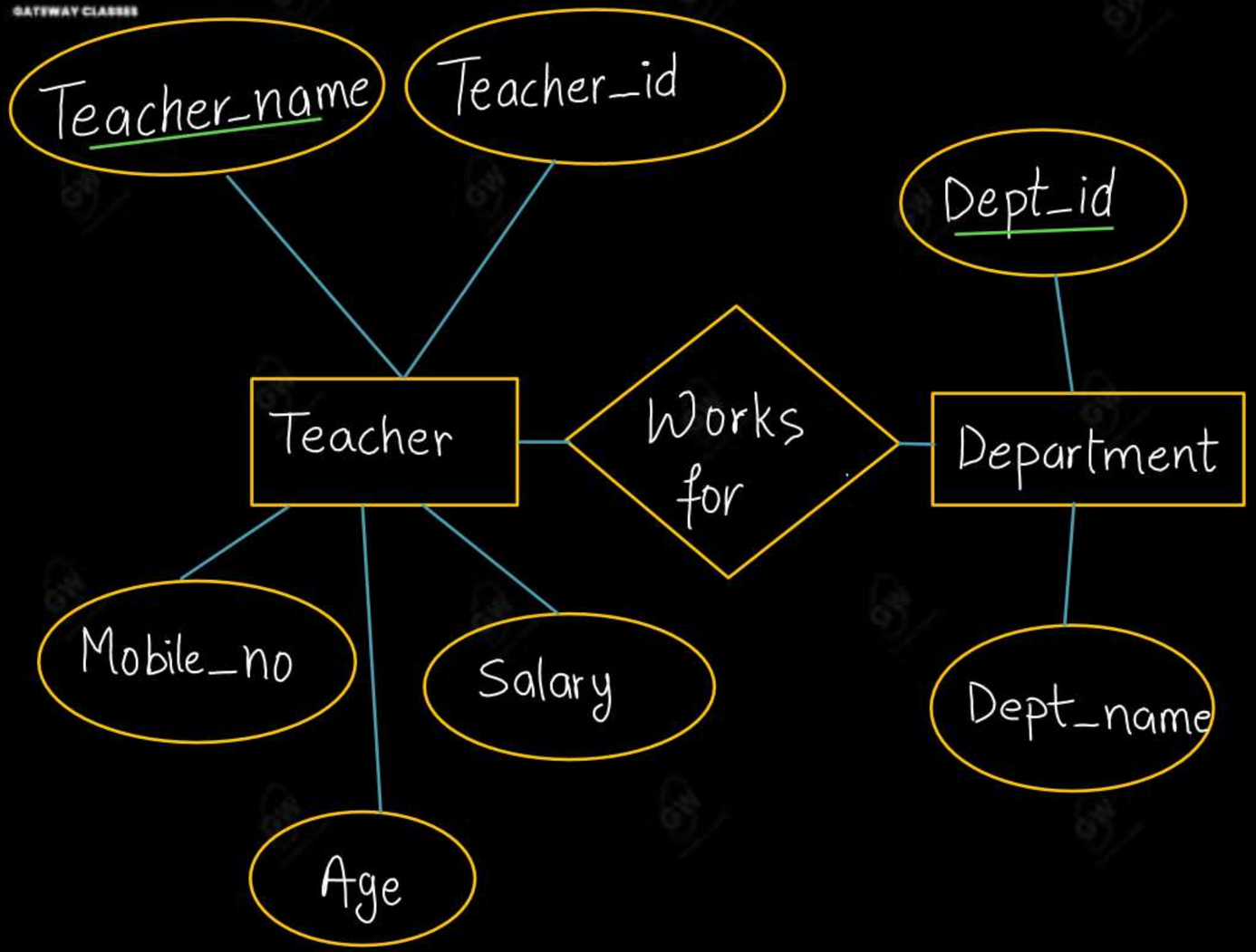
| Types of Data Model | Description | Structure | Advantages | Disadvantages |
|---|---|---|---|---|
| 1. Hierarchical Model | Organizes data in a tree-like structure. Each child has only one parent, but a parent can have multiple children (one-to-many relationship). | Tree (Parent → Child) | ➢ Simple and fast for hierarchical data<br>➢ Easy to navigate | ➢ Cannot handle many-to-many relationships<br>➢ Difficult to modify structure |
| 2. Network Model | Similar to hierarchical but allows many-to-many relationships. Data is organized using graph-like structures with pointers. | Graph (Record and Sets) | ➢ Flexible and fast<br>➢ Supports complex relationships | ➢ Complex to design and maintain<br>➢ Requires navigational access |

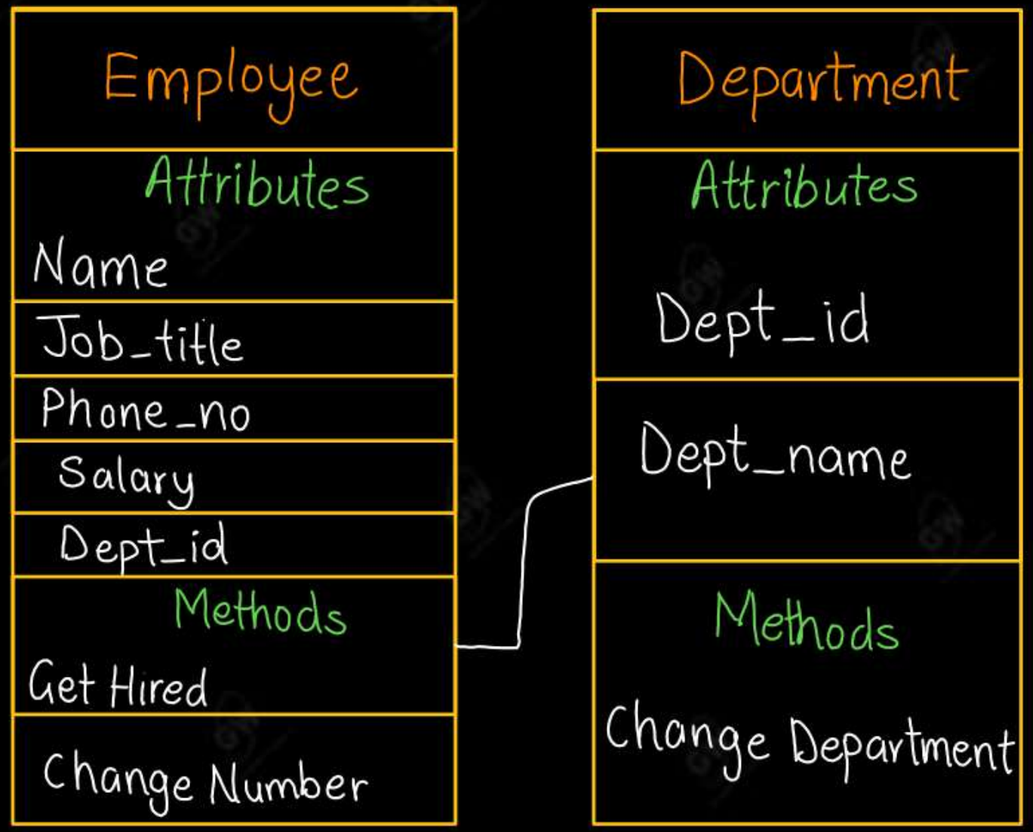
| Types of Data Model | Description | Structure | Advantages | Disadvantages |
|---|---|---|---|---|
| 3. Relational Model | Organizes data into tables (relations) with rows (tuples) and columns (attributes). Uses SQL to manipulate data. | Tables (Rows and Columns) | Easy to use and query<br>Highly flexible and scalable | Slower for very large data<br>Requires normalization |
| 4. Entity-Relationship (ER) Model | A conceptual model that represents data using entities, attributes, and relationships. Used for database design, not storage. | Diagrams (ER Diagrams) | Easy to visualize data<br>Good for designing relational databases | Not used for storage<br>Cannot perform queries directly |

| Data Model | Description | Structure | Advantages | Disadvantages |
|---|---|---|---|---|
| 5. Object-Oriented Model | Stores data as objects (like in OOP), which include attributes (data) and methods (behavior). Supports inheritance, encapsulation. | Objects and Classes | ➢ Matches object-oriented programming<br>➢ Supports complex data types | Less mature, Limited support in traditional RDBMS |
| 6. Document / NoSQL Model | Stores data as documents, usually in JSON or XML format. Supports semi-structured or unstructured data. Mostly used in NoSQL databases. | Collections of Documents | ➢ High performance<br>➢Scalable for big data and real-time apps | ➢ No strict schema Weaker consistency and integrity control |


Shoes
Women's shoes
Men's Shoes
High Heels
Bellies
Sports Shoes
Sneakers


College
CSE Department
Library
Student


Teacher_name
Teacher_id
Dept_id
Teacher
Works for
Department
Mobile_no
Salary
Dept_name
Age


Employee
Attributes
Name
Job_title
Phone_no
Salary
Dept_id
Methods
Get Hired
Change Number
Department
Attributes
Dept_id
Dept_name
Methods
Change Department

# What is Database Schema? [AKTU 2017-18/2022-23]

A database schema is the structure or blueprint of a database.
It defines how data is organized, including:
Tables, Columns, Data types, Keys (Primary, Foreign)
Relationships
It tells the DBMS what the database should look like — like a design map

| Type of Schema | Description | Example |
|---|---|---|
| 1. Physical Schema | Describes how data is actually stored on the storage device (files, indexes). | Data stored in .dat files with indexing like B-trees. |
| 2. Logical Schema | Describes tables, fields, relationships, constraints — what data is stored. | Table: STUDENT(roll_no, name, marks)Primary Key: roll_no |
| 3. View Schema (External Schema) | Defines how users see the data (customized view). Hides irrelevant data. | A student can only see their marks; an admin sees all student records. |

# What is a Key in DBMS? [AKTU 2020-21/ 2021-22]

It is an attribute or a set of attributes that is used to:

- Uniquely identify rows (tuples) in a table.
- Maintain relationships between different tables.
- Ensure data integrity.

| Key Type | Description | Example | Purpose |
|---|---|---|---|
| 1. Primary Key | A column or group of columns that uniquely identifies each row in a table. | Roll_No in STUDENT(roll_no, name, marks)Each roll_no is unique. | Avoid duplicate records; cannot be NULL. |
| 2. Candidate Key | All possible columns that can uniquely identify a row. One becomes primary key. | roll_no, Aadhar_no — both uniquely identify students. | Choose best option as primary key. |
| 3. Super Key | A set of attributes that uniquely identifies a row (may include extra attributes). | roll_no + name (even though roll_no alone is enough). | Superset of candidate key. |

| Key Type | Description | Example | Purpose |
| --- | --- | --- | --- |
| 4. Foreign Key | A column that refers to primary key of another table to create relationship. | student_id in Marks table refers to student_id in Student table. | Maintains referential integrity. |
| 5. Composite Key | A primary key made of two or more columns. | order_id + product_id in OrderDetails table uniquely identify a row. | Used when no single column is unique. |
| 6. Alternate Key | A candidate key not chosen as the primary key. | If roll_no is primary key, then Aadhar_no is alternate key. | Backup option for uniqueness. |
| 7. Unique Key | Ensures unique values in a column, but can have one NULL. | email in a user table — all must be different. | Enforces uniqueness with NULL allowed once. |

# ER diagram

## AKTU 2022-23/2023-24

- ER diagram or Entity Relationship diagram is a conceptual model that gives the graphical representation of the logical structure of the database.
- It shows all the constraints and relationships that exist among the different components.

## Components of ER diagram-

An ER diagram is mainly composed of following three components

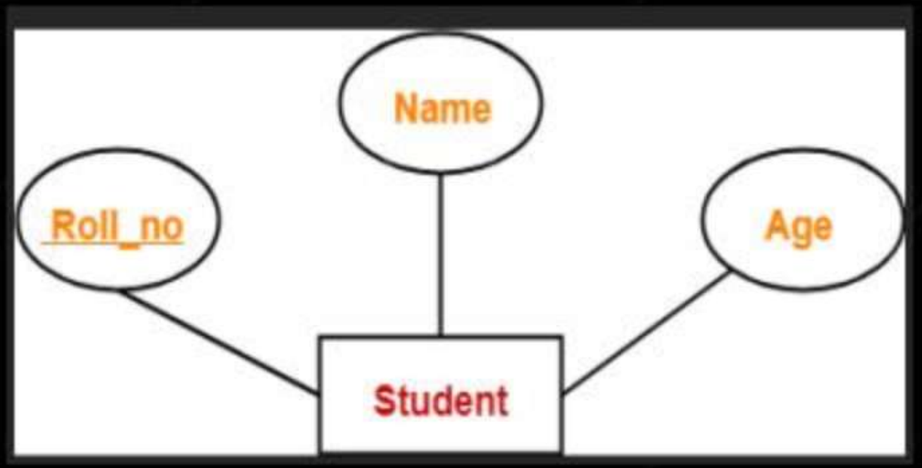
Entity Sets, Attributes, Relationship Set

consider the following Student table-

| Roll_no | Name | Age |
|---|---|---|
| 1 | Akshay | 20 |
| 2 | Rahul | 19 |
| 3 | Pooja | 20 |
| 4 | Aarti | 19 |

This complete table is referred to as “Student Entity Set” and each row represents an “entity”.

## Representation as ER Diagram-




### 1. Entity Set in DBMS-

An entity set is a set of same type of entities.

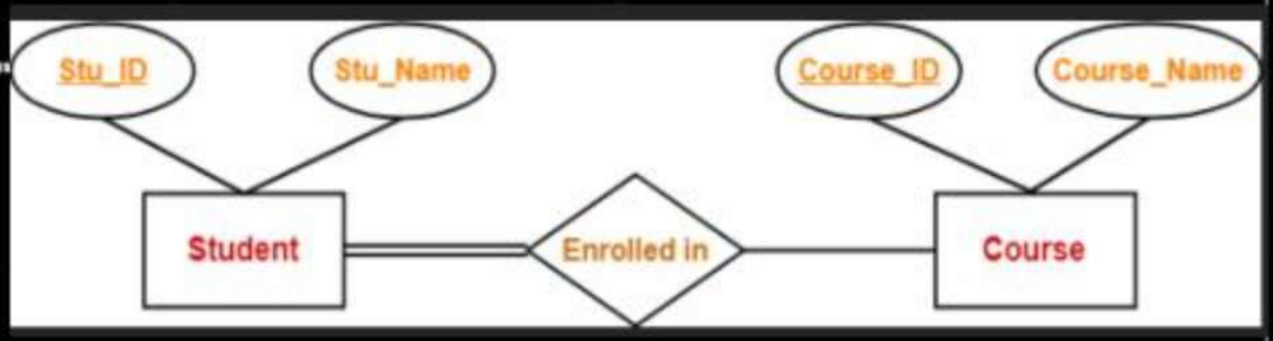
An entity refers to any object having-

- Either a physical existence such as a particular person, office, house or car.
- Or a conceptual existence such as a school, a university, a company or a job.

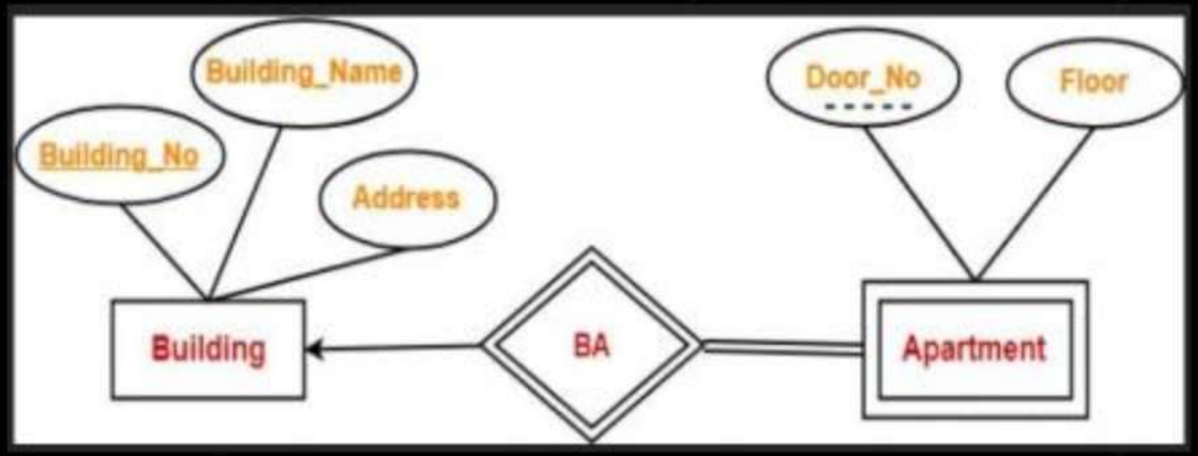
## Types of Entity Sets-

### 1.1. Strong Entity Set

- A strong entity set is an entity set that contains sufficient attributes to uniquely identify all its entities.
- In other words, a primary key exists for a strong entity set.
- Primary key of a strong entity set is represented by underlining it.



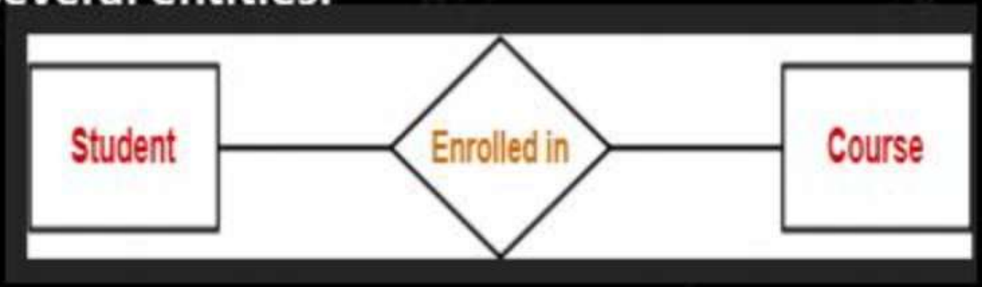

## 1.2.Weak Entity Set-




- A weak entity set is an entity set that does not contain sufficient attributes to uniquely identify its entities.
- In other words, a primary key does not exist for a weak entity set.
- However, it contains a partial key called as a discriminator.
- Discriminator can identify a group of entities from the entity set.
- Discriminator is represented by underlining with a dashed line.

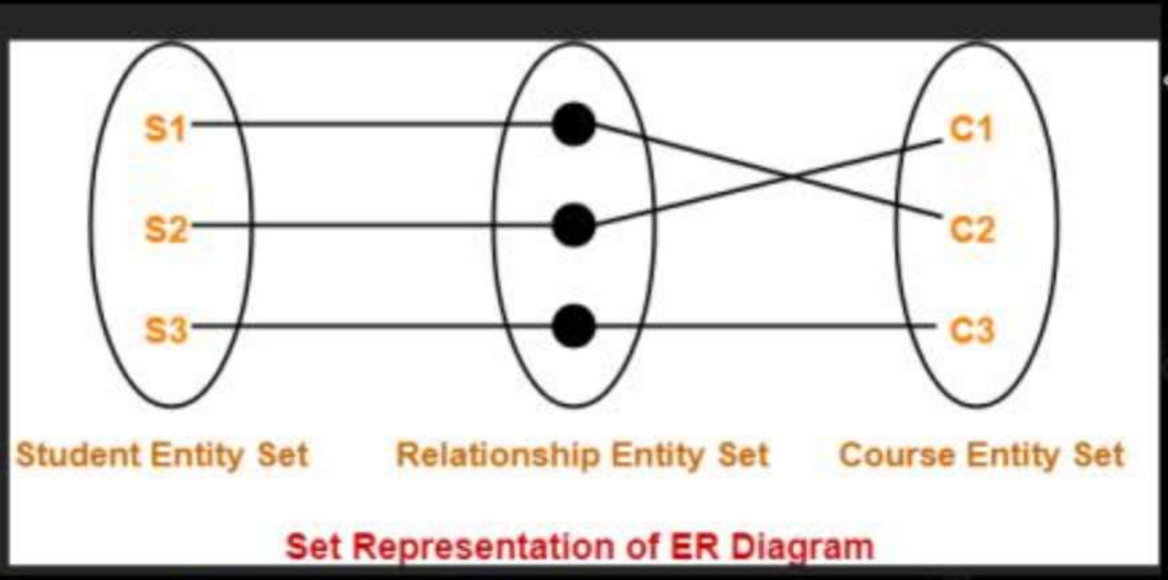
# 2.Relationship in DBMS-

A relationship is defined as an association among several entities.




## Relationship Set-

A relationship set is a set of relationships of same type.

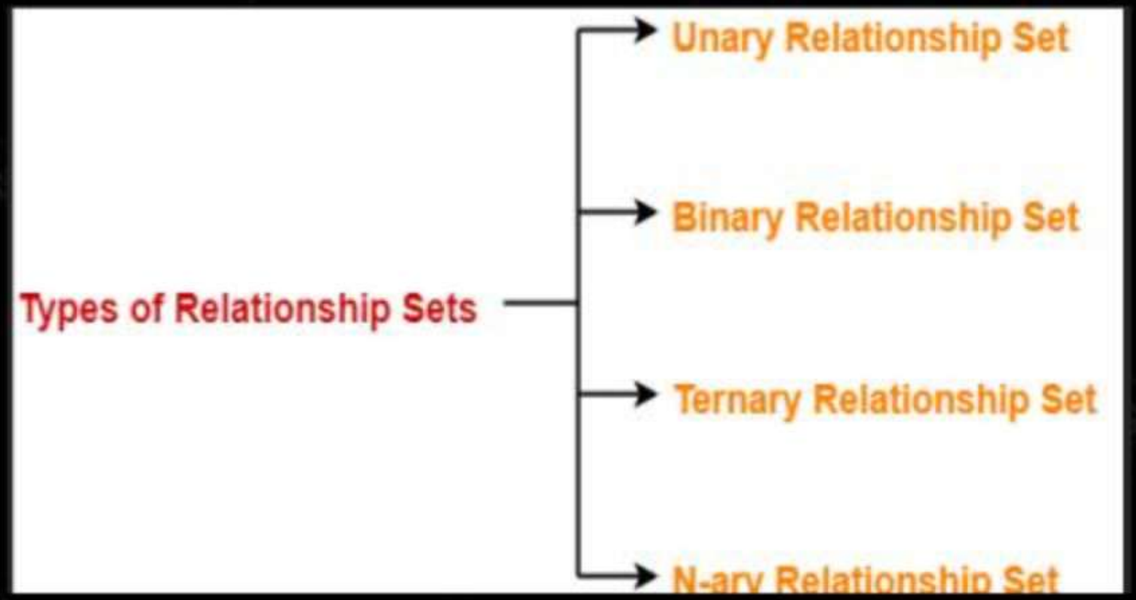



Set Representation of ER Diagram

## Degree of a Relationship Set-

The number of entity sets that participate in a relationship set is termed as the degree of that relationship set.

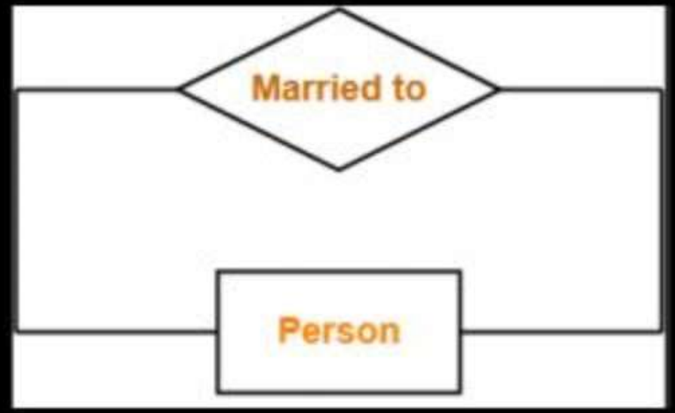
# Types of relationship set




## 2.1. Unary Relationship Set-

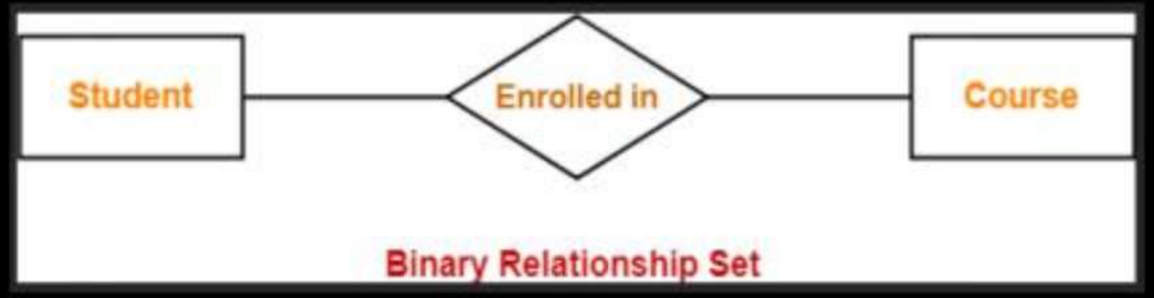
Unary relationship set is a relationship set where only one entity set participates in a relationship set.

One person is married to only one person



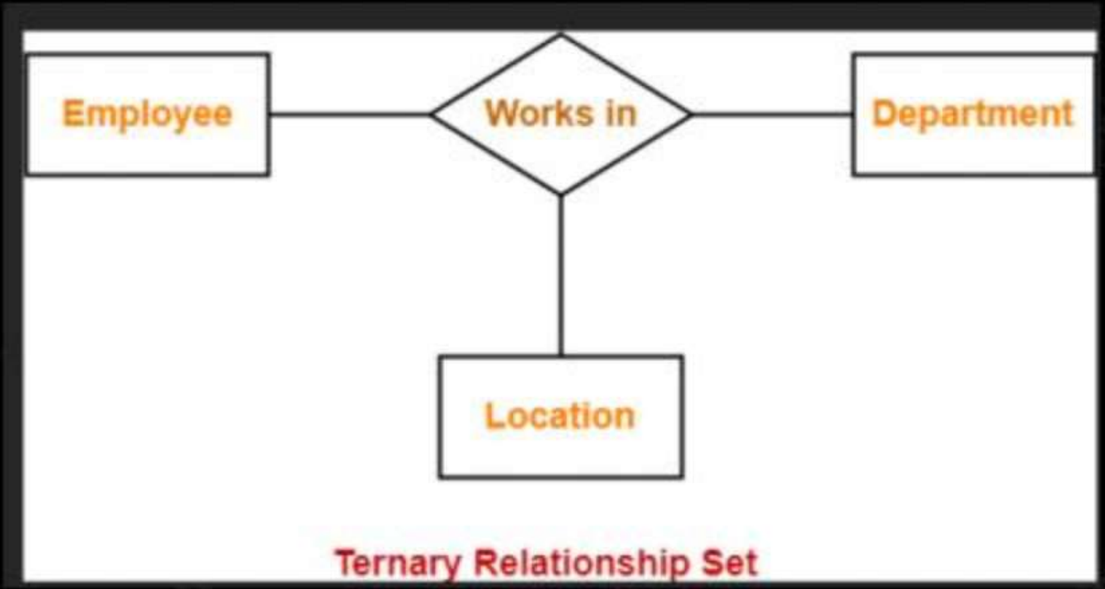

## 2.2.Binary Relationship Set

Binary relationship set is a relationship set where two entity sets participate in a relationship set



## 2.3. Ternary Relationship Set-

Ternary relationship set is a relationship set where three entity sets participate in a relationship set.




Ternary Relationship Set

## 2.4. N-ary Relationship Set-

N-ary relationship set is a relationship set where ‘n’ entity sets participate in a relationship set.

# Attributes in ER Diagram-

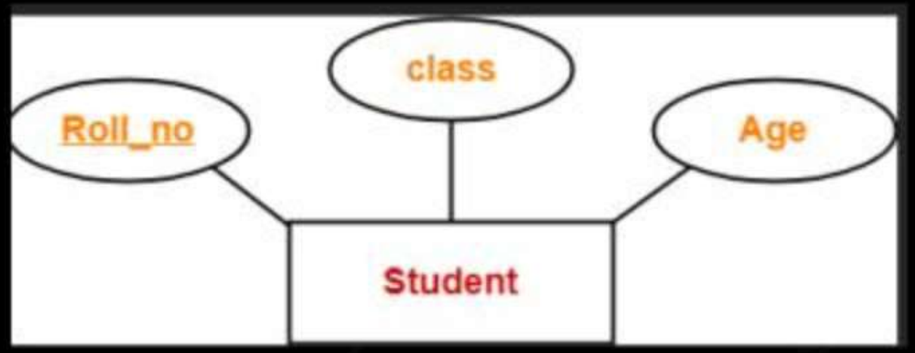
➢Attributes are the descriptive properties which are owned by each entity of an Entity Set.

➢There exist a specific domain or set of values for each attribute from where the attribute can take its values.

## Types of Attributes

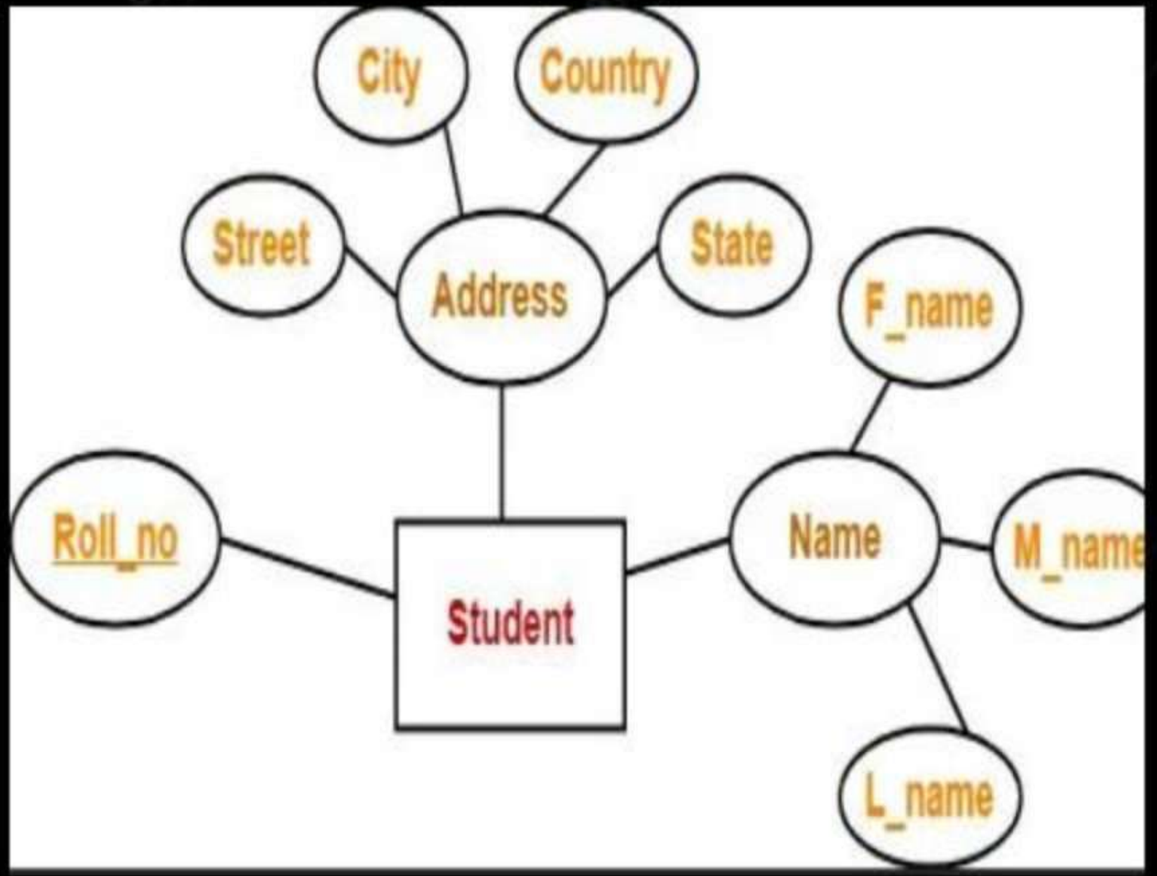
### 1. Simple Attributes-

Simple attributes are those attributes which can not be divided further



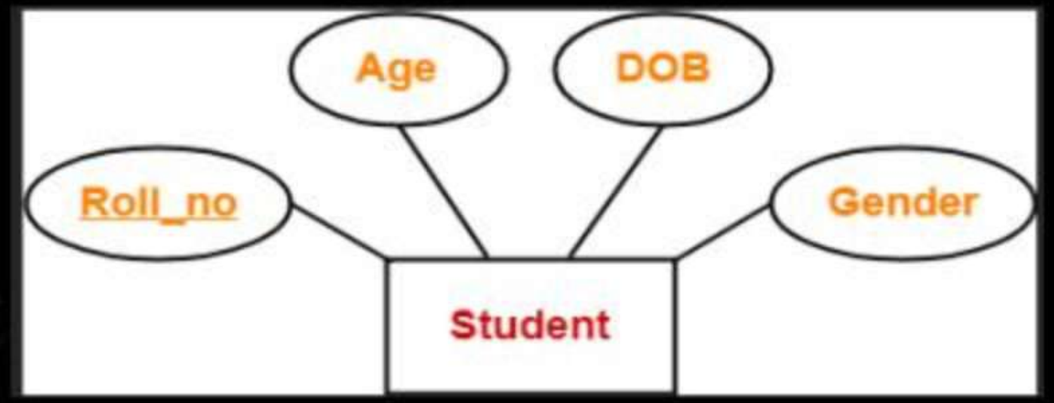

### 2. Composite Attributes-

Composite attributes are those attributes which are composed of many other simple attributes.



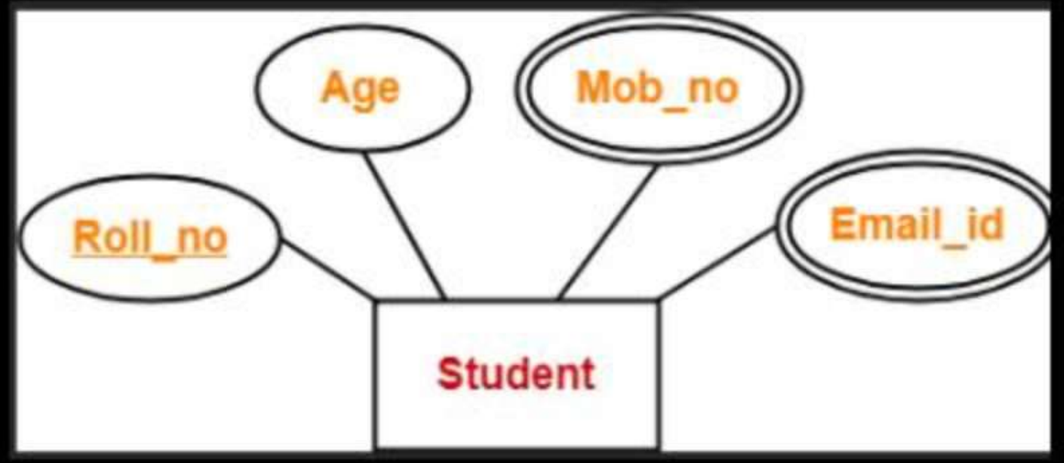
## 3. Single Valued Attributes-

Single valued attributes are those attributes which can take only one value for a given entity from an entity set.



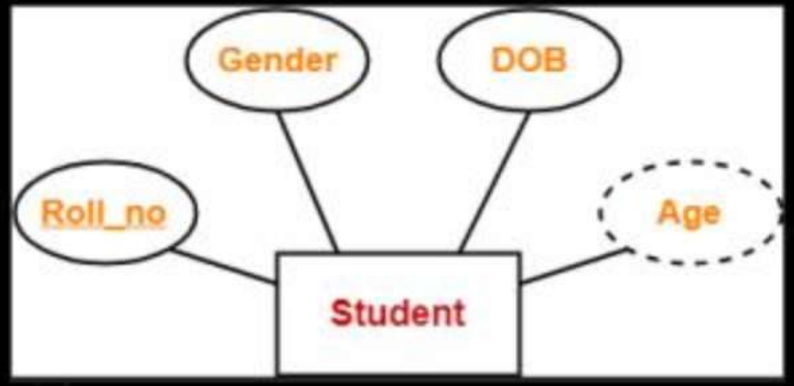

## 4. Multi Valued Attributes-

Multi valued attributes are those attributes which can take more than one value for a given entity from an entity set.



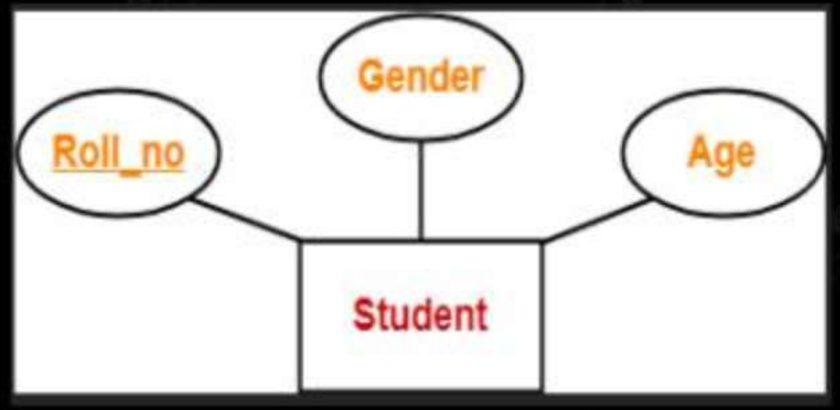

## 5. Derived Attributes-

Derived attributes are those attributes which can be derived from other attribute(s).



## 6. Key Attributes-

Key attributes are those attributes which can identify an entity uniquely in an entity set.




## Types of constraints

1.Cardinality ratio constraints

2.Participation constraints

### 1.Cardinality constraint

It defines the maximum number of relationship instances in which an entity can participate

There are 4 types of cardinality ratios-

1.1.Many-to-Many cardinality (m:n)

1.2.Many-to-One cardinality (m:1)

1.3.One-to-Many cardinality (1:n)

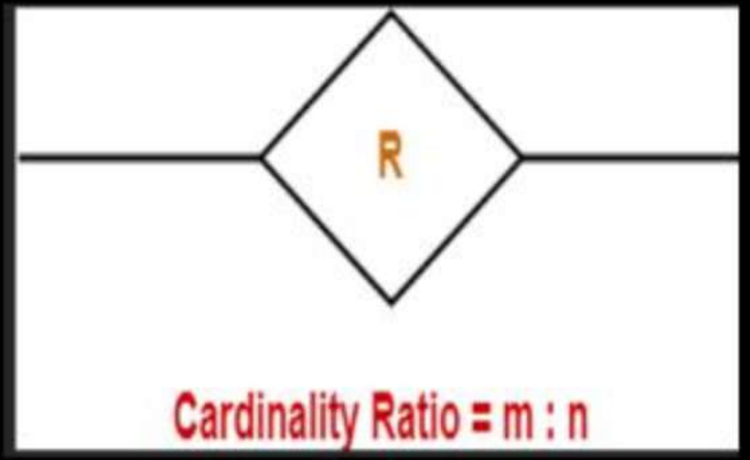
1.4.One-to-One cardinality (1:1 )

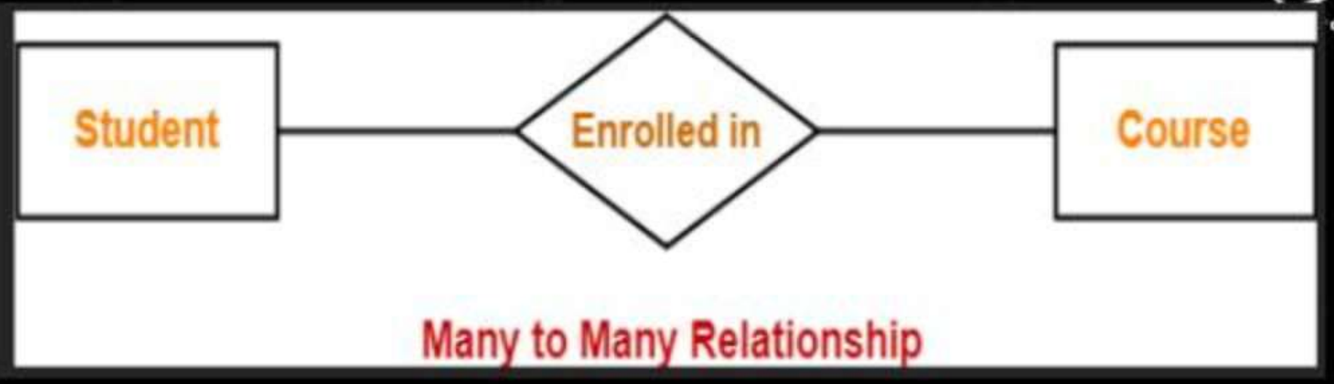
## 1. 1.Many-to-Many Cardinality-

By this cardinality constraint,

➢An entity in set A can be associated with any number (zero or more) of entities in set B.

➢An entity in set B can be associated with any number (zero or more) of entities in set A.




➢One student can enroll in any number (zero or more) of courses.

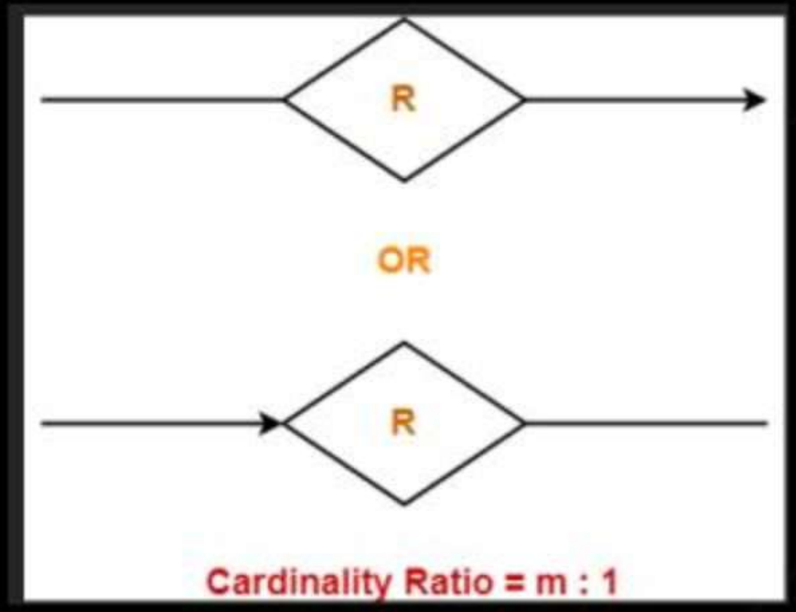
➢One course can be enrolled by any number (zero or more) of student

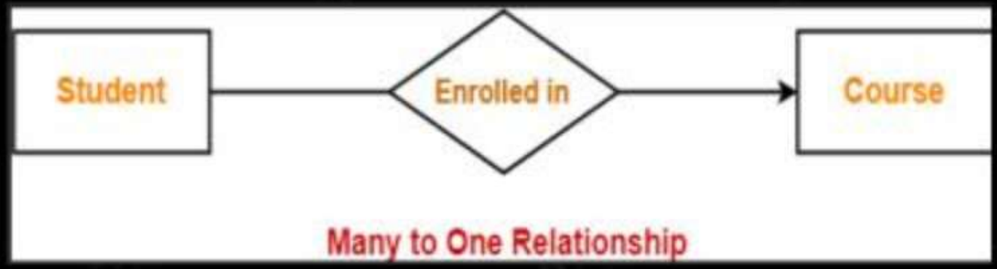
## 1.2. Many-to-One Cardinality-

By this cardinality constraint,

➢An entity in set A can be associated with at most one entity in set B.

➢An entity in set B can be associated with any number (zero or more) of entities in set A.




➢One student can enroll in at most one course.

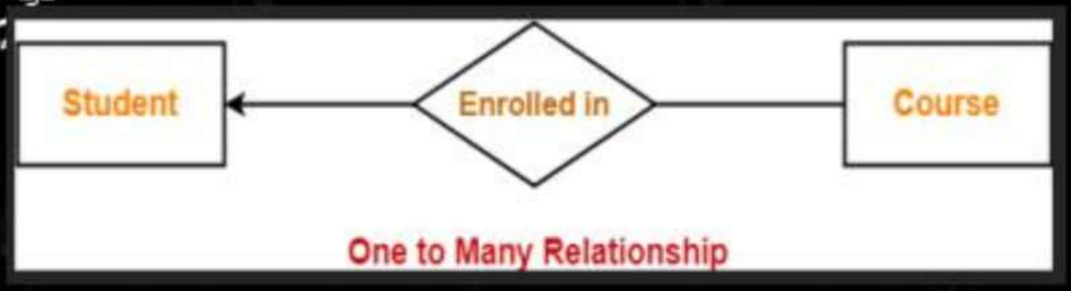
➢One course can be enrolled by any number (zero or more) of students.

## 1.3. One-to-Many Cardinality-

By this cardinality constraint,

➢An entity in set A can be associated with any number (zero or more) of entities in set B.

➢An entity in set B can be associated with at most one entity in set A




➢One student can enroll in any number (zero or more) of courses.

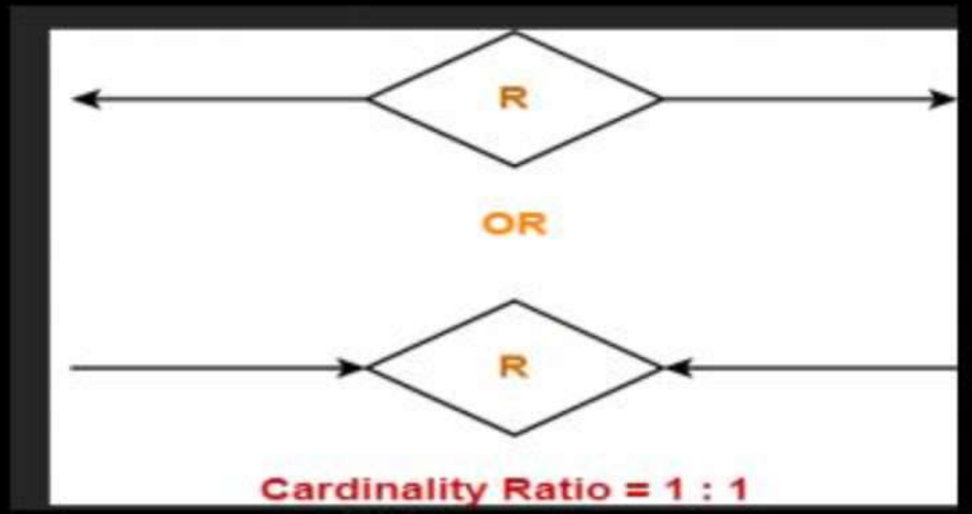
➢One course can be enrolled by at most one student.

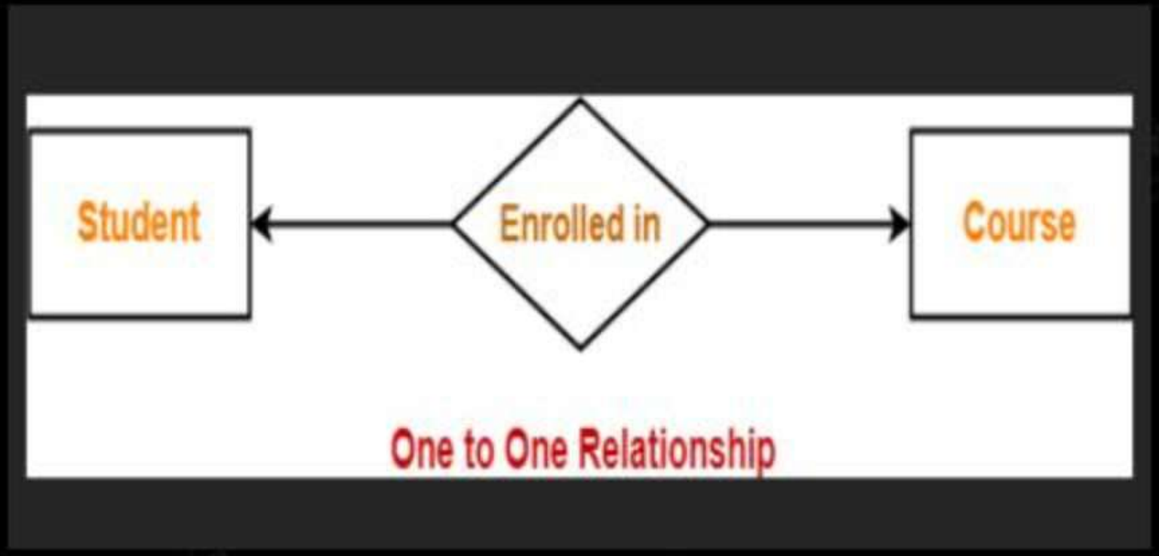
### 1.4. One-to-One Cardinality-

By this cardinality constraint,

➢An entity in set A can be associated with at most one entity in set B.

➢An entity in set B can be associated with at most one entity in set A.



➢one student can enroll in at most one course.

➢One course can be enrolled by at most one student.

## 2.Participation Constraints-

Participation constraints define the least number of relationship instances in which an entity must compulsorily participate

## Types of Participation Constraints-

### 2.1. Total Participation-

➢It specifies that each entity in the entity set must compulsorily participate in at least one relationship instance in that relationship set.

➢That is why, it is also called as mandatory participation.

➢Total participation is represented using a double line between the entity set and relationship set.






➢Double line between the entity set "Student" and relationship set "Enrolled in" signifies total participation.

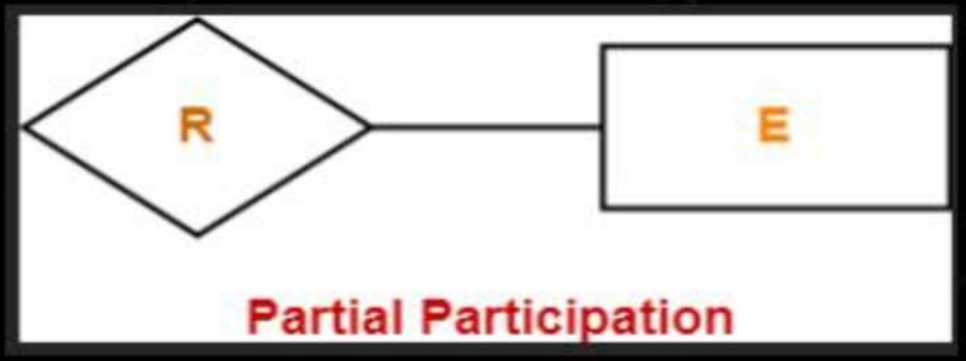
➢It specifies that each student must be enrolled in at least one course

## 2.2 Partial Participation

➢It specifies that each entity in the entity set may or may not participate in the relationship instance in that relationship set.

➢That is why, it is also called as optional participation.

➢Partial participation is represented using a single line between the entity set and relationship set





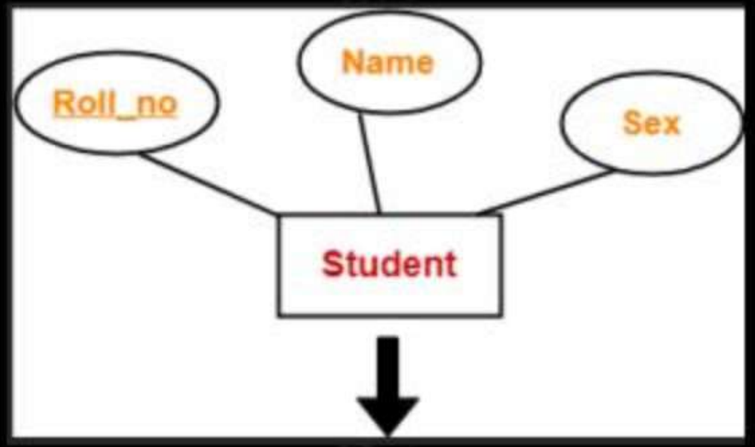

- Single line between the entity set “Course” and relationship set “Enrolled in” signifies partial participation.
- It specifies that there might exist some courses for which no enrollments are made

## Converting ER Diagrams to Tables-

- After designing an ER Diagram
- ER diagram is converted into the tables in relational model.
- This is because relational models can be easily implemented by RDBMS like MySQL , Oracle etc.

**Rule-01:** For Strong Entity Set With Only Simple Attributes-

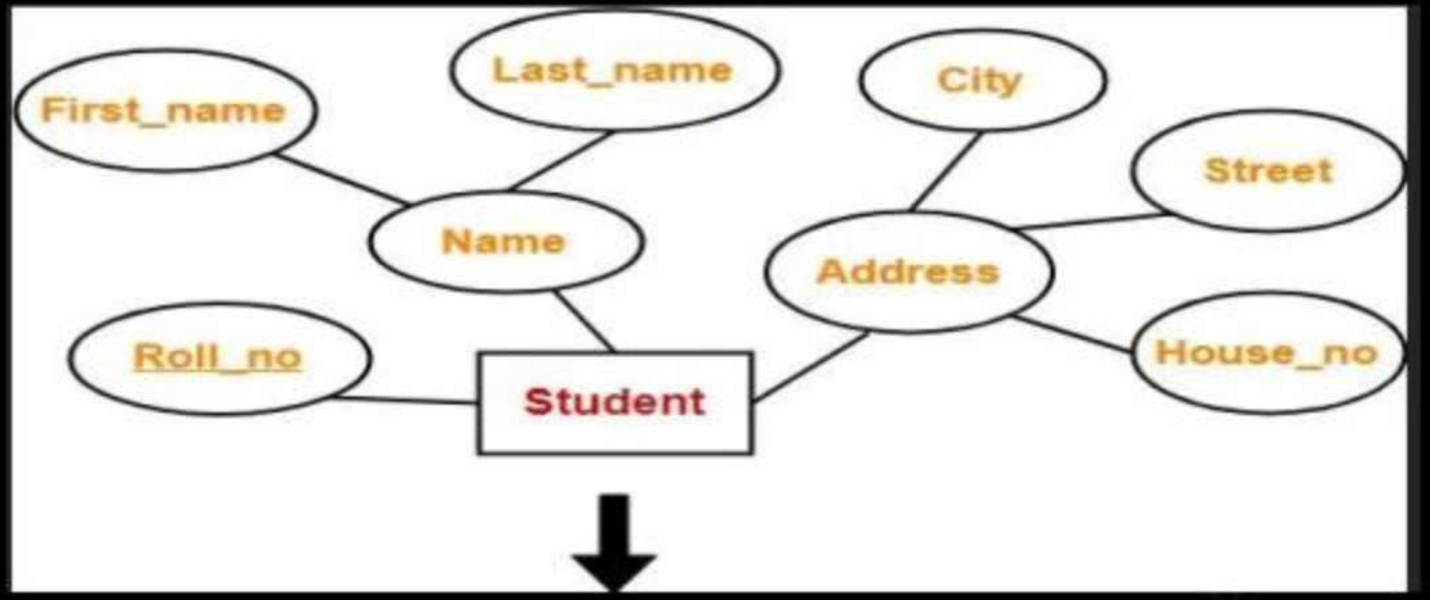
- A strong entity set with only simple attributes will require only one table in relational model.
- Attributes of the table will be the attributes of the entity set.
- The primary key of the table will be the key attribute of the entity se




| Roll_no | Name | Sex |
|---|---|---|
| | | |

## Rule-02: For Strong Entity Set With Composite Attributes-

- A strong entity set with any number of composite attributes will require only one table in relational model.
- While conversion, simple attributes of the composite attributes are taken into account and not the composite attribute itself.

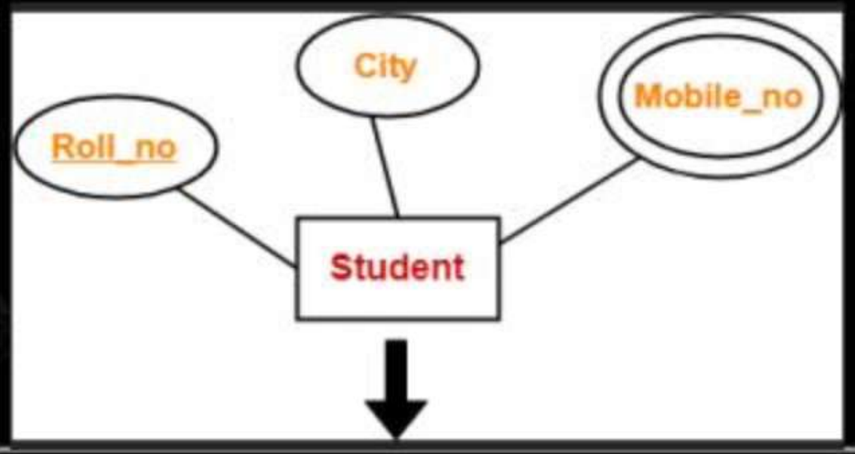



| Roll_no | First_name | Last_name | House_no | Street | City |
|---|---|---|---|---|---|
| | | | | | |

Schema : Student ( Roll_no , First_name , Last_name , House_no , Street , City )

## Rule-03: For Strong Entity Set With Multi Valued Attributes-

- A strong entity set with any number of multi valued attributes will require two tables in relational model.
- One table will contain all the simple attributes with the primary key.
- Other table will contain the primary key and all the multi valued attributes.

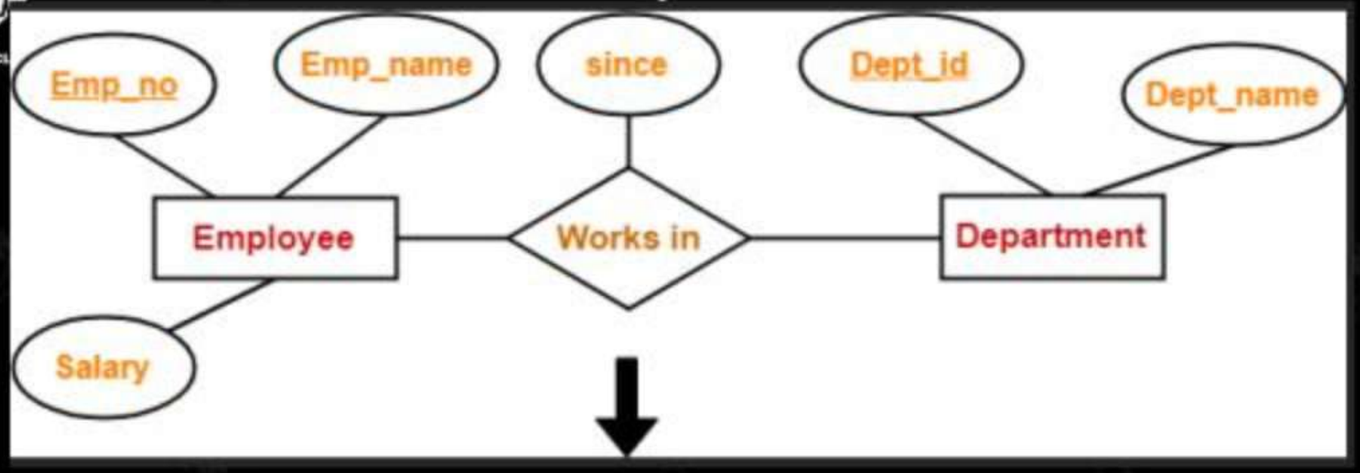



| Roll_no | City |
|---|---|
| | |

| Roll_no | Mobile_no |
|---|---|
| | |

## Rule-04: Translating Relationship Set into a Table-

- A relationship set will require one table in the relational model.
- Attributes of the table are-
- Primary key attributes of the participating entity sets
- Its own descriptive attributes if any.
- Set of non-descriptive attributes will be the primary key.




| Emp_no | Dept_id | since |
|---|---|---|
| | | |

Schema : Works in ( Emp_no , Dept_id , since )

If we consider the overall ER diagram, three tables will be required in relational model Employee,Department,Works in

## Rule-05: For Binary Relationships With Cardinality Ratios-

The following four cases are possible-

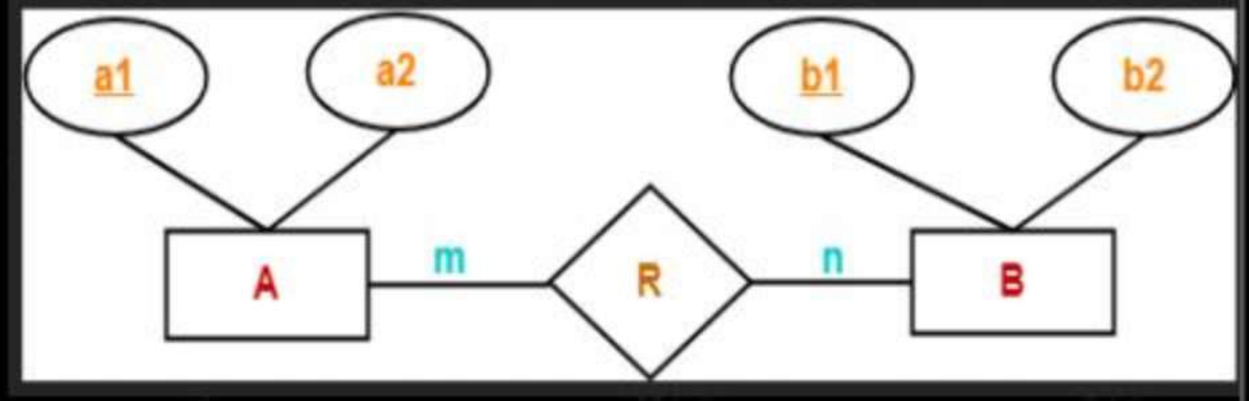
**Case-01:** Binary relationship with cardinality ratio m:n

**Case-02:** Binary relationship with cardinality ratio 1:n

**Case-03:** Binary relationship with cardinality ratio m:1

**Case-04:** Binary relationship with cardinality ratio 1:1

## Case-01: For Binary Relationship With Cardinality Ratio m:n



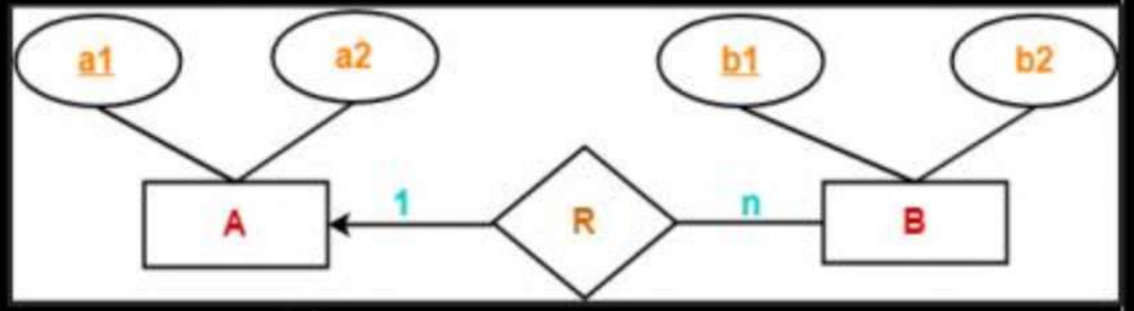

Here, three tables will be required-

A ( a1 , a2 )

R ( a1 , b1 )

B ( b1 , b2 )

## Case-02: For Binary Relationship With Cardinality Ratio 1:n

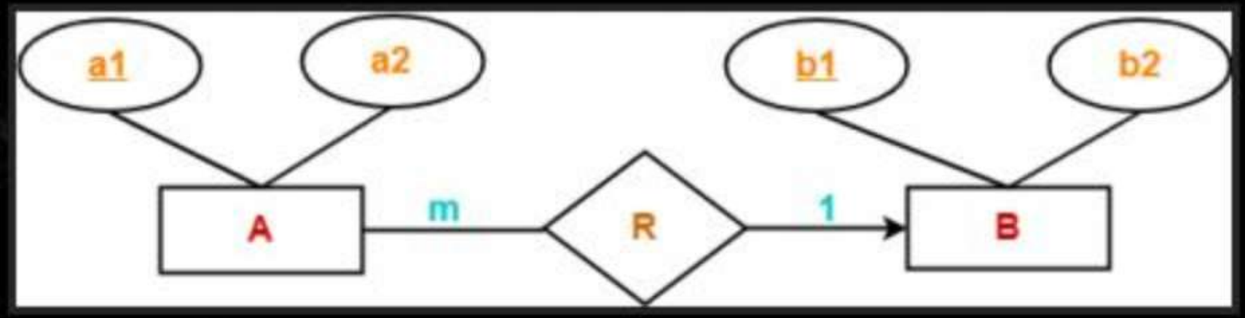



Here, two tables will be required-

A ( a1 , a2 )

BR ( a1 , b1 , b2 )

## Case-03: For Binary Relationship With Cardinality Ratio m:1

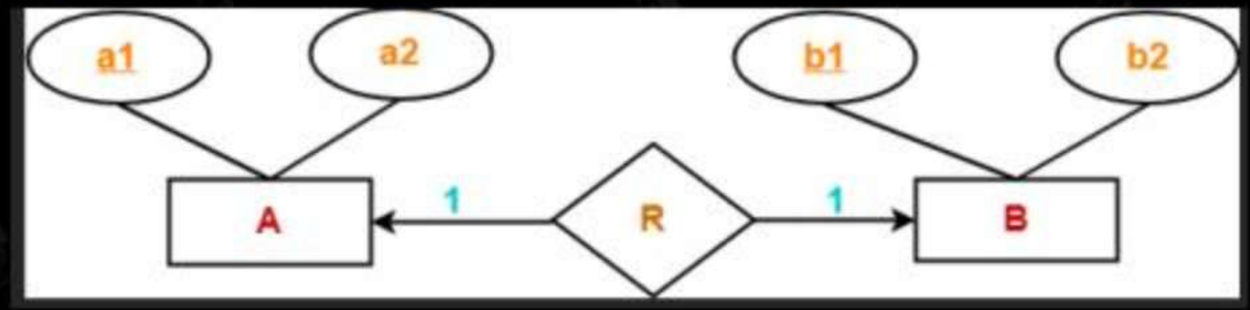



Here, two tables will be required-

AR ( a1 , a2 , b1 )

B ( b1 , b2 )

## Case-04: For Binary Relationship With Cardinality Ratio 1:1




**Way-01:**

AR ( a1 , a2 , b1 )

B ( b1 , b2 )

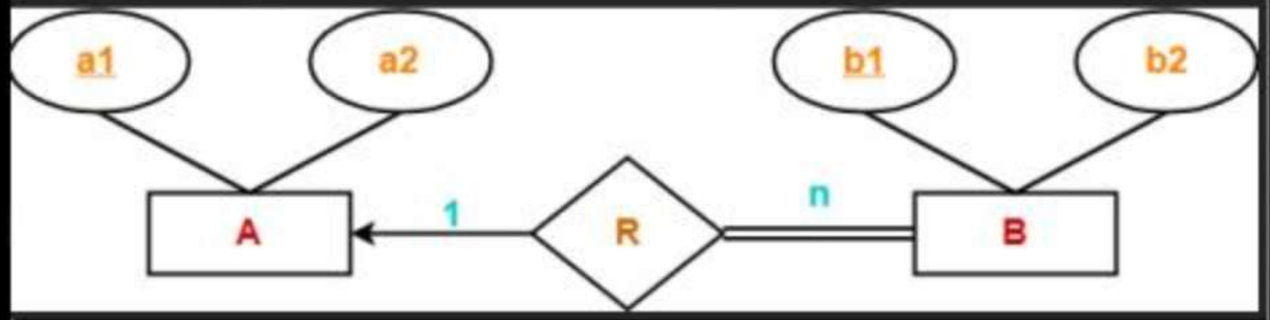
**Way-02:**

A ( a1 , a2 )

BR ( a1 , b1 , b2 )

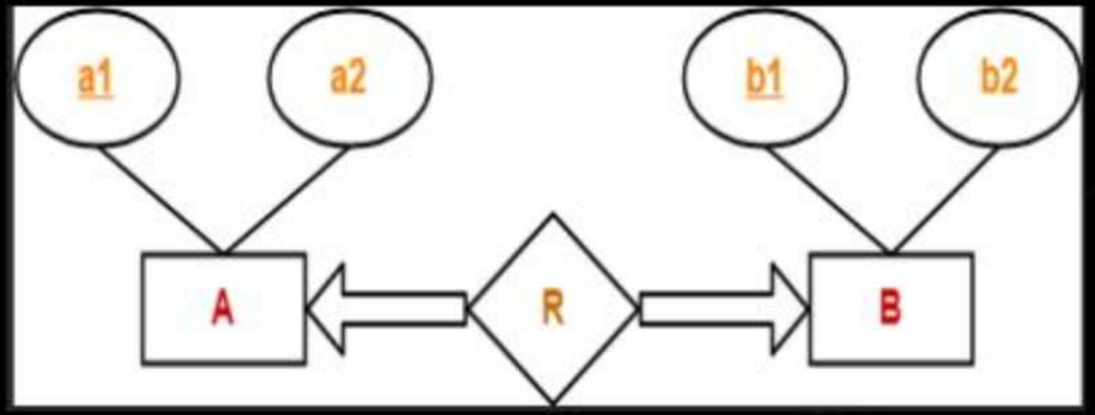
## Rule-06: For Binary Relationship With Both Cardinality Constraints and Participation Constraints-

**Case-01:** For Binary Relationship With Cardinality Constraint and Total Participation Constraint From One Side-



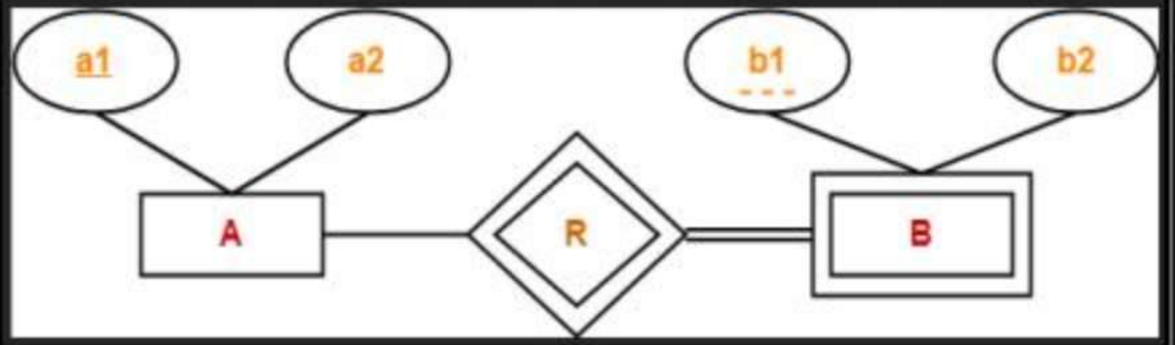

Then, two tables will be required-
A ( a1 , a2 )
BR ( a1 , b1 , b2 )

**Case-02:** For Binary Relationship With Cardinality Constraint and Total Participation Constraint From Both Sides-




ARB ( a1 , a2 , b1 , b2 )

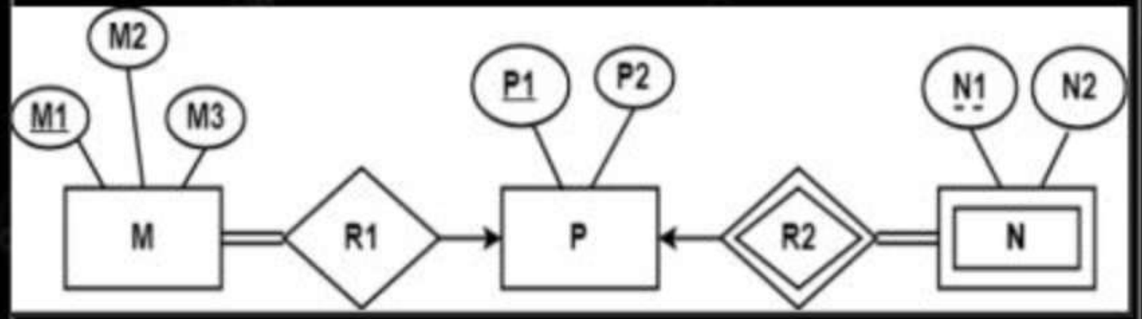
## Rule-07: For Binary Relationship With Weak Entity Set




A ( a1 , a2 )
BR ( a1 , b1 , b2 )

# PROBLEMS BASED ON CONVERTING ER DIAGRAM TO TABLES-

**1. Find the minimum number of tables required to represent the given ER diagram in relational model-**

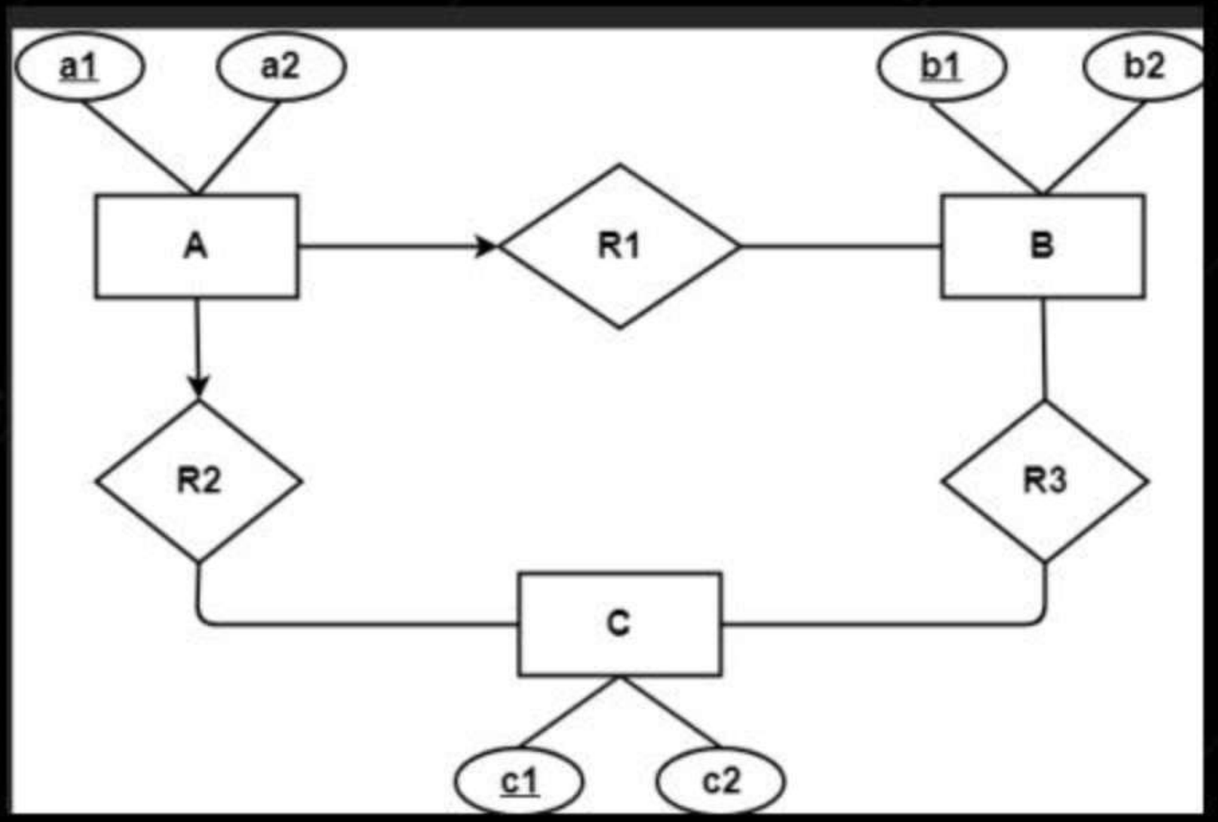



MR1 (M1 , M2 , M3 , P1)

P (P1 , P2)

NR2 (P1 , N1 , N2)

**2.**



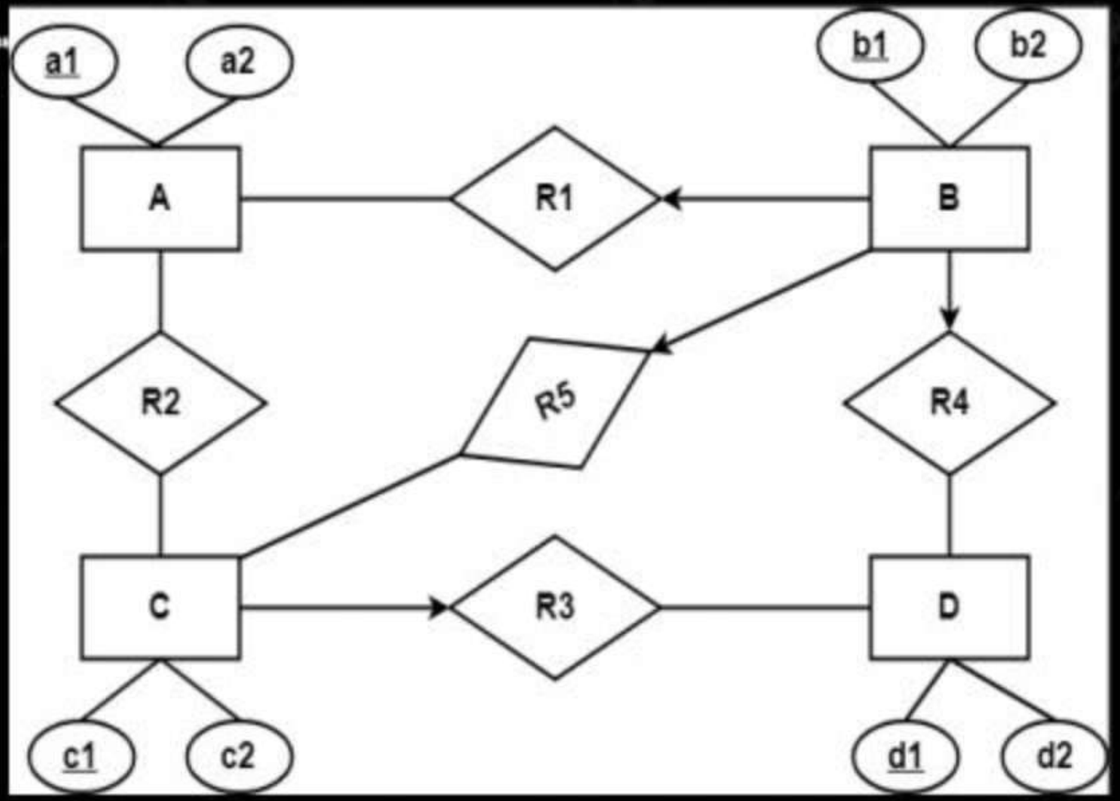

AR1R2 (a1 , a2 , b1 , c1)

B (b1 , b2)

C (c1 , c2)

R3 (b1 , c1)

3.




BR1R4R5 (b1 , b2 , a1 , c1 , d1)

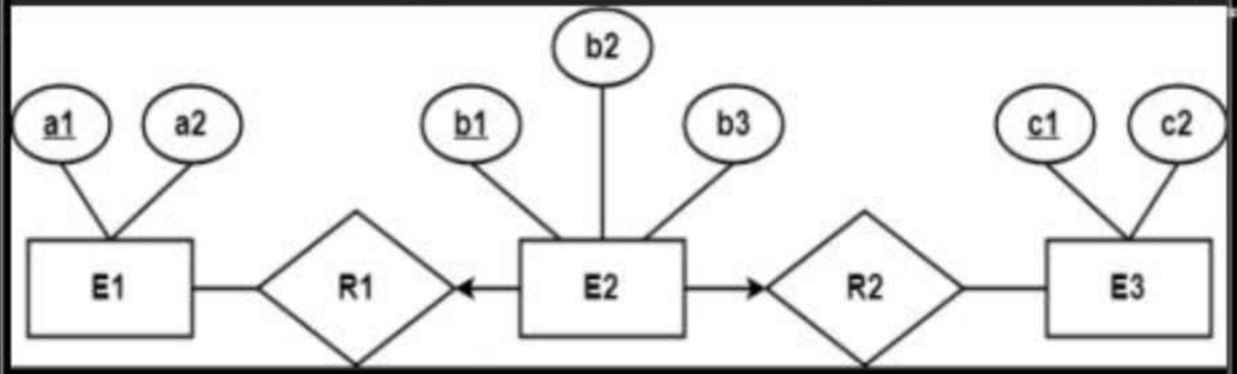
A (a1 , a2)

R2 (a1 , c1)

CR3 (c1 , c2 , d1)

D (d1 , d2)

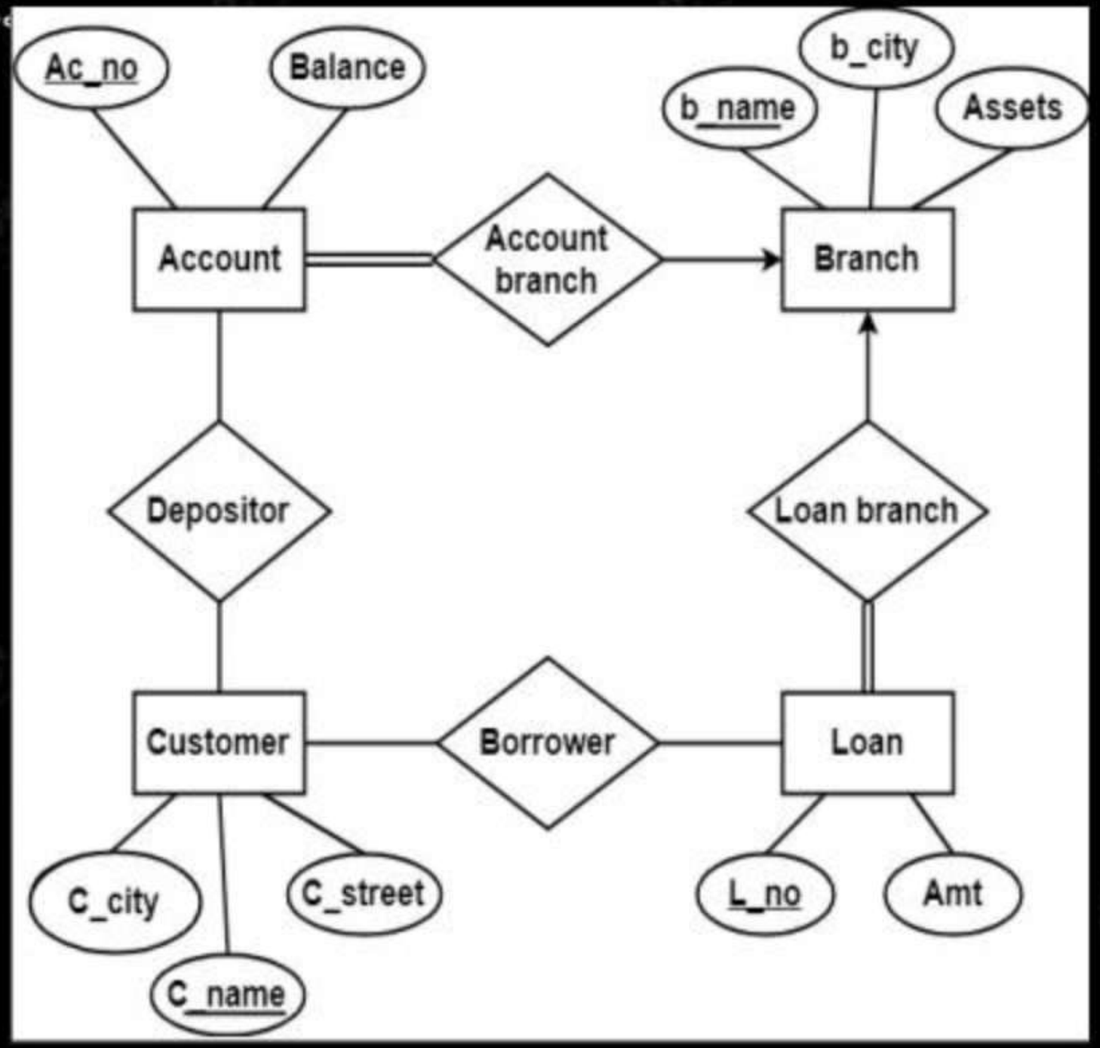
4.




E1 (a1 , a2)

E2R1R2 (b1 , b2 , a1 , c1 , b3)

E3 (c1 , c2)




Account (Ac_no , Balance , b_name)

Branch (b_name , b_city , Assets)

Loan (L_no , Amt , b_name)

Borrower (C_name , L_no)

Customer (C_name , C_street , C_city)

Depositor (C_name , Ac_no)

# Difference between AKTU 2018-19

| Feature | DBMS (Database Management System) | RDBMS (Relational Database Management System) |
|---|---|---|
| 1. Data Storage | Stores data as files or collections of data | Stores data in the form of tables (rows and columns) |
| 2. Data Relationship | Does not support relationships between data | Supports relationships using foreign keys and primary keys |
| 3. Data Integrity | Limited data integrity and constraints | Enforces data integrity using constraints like PK, FK, Not Null, etc. |
| 4. Normalization | Normalization not supported | Supports normalization to reduce redundancy |
| 5. Security | Basic level of security | Advanced security features (user roles, access control) |
| 6. Examples | Microsoft Access, XML, File System | MySQL, Oracle, SQL Server, PostgreSQL |
| 7. Multi-user support | Usually supports only single user | Designed to support multiple users concurrently |

## Difference between

| Feature | DBMS (Database Management System) | RDBMS (Relational Database Management System) |
|---|---|---|
| 8. Transactions | No proper support for ACID properties | Follows ACID properties for transactions |
| 9. Query Language | May not support SQL | Uses Structured Query Language (SQL) |
| 10. Data Redundancy | High data redundancy due to no relational model | Low redundancy through relationships and normalization |

# What is a DBMS Interface?

A DBMS interface is the way users interact with the database system.
It allows users (end users, programmers, or administrators) to access, modify, and manage data stored in the database.

## Type of interface

| Interface Type | Description | Used By | Example |
|---|---|---|---|
| 1. Menu-based Interface | Provides options in the form of menus for users to select commands without typing queries. | End users (non-technical) | ATMs, hotel booking systems |
| 2. Form-based Interface | Uses forms with labeled fields for data entry or retrieval. | Clerical or data-entry users | Student registration form |
| 3. Graphical User Interface (GUI) | Uses windows, icons, buttons, and visual elements to interact with the database. | General users | Microsoft Access GUI |

# Types of DBMS Interfaces

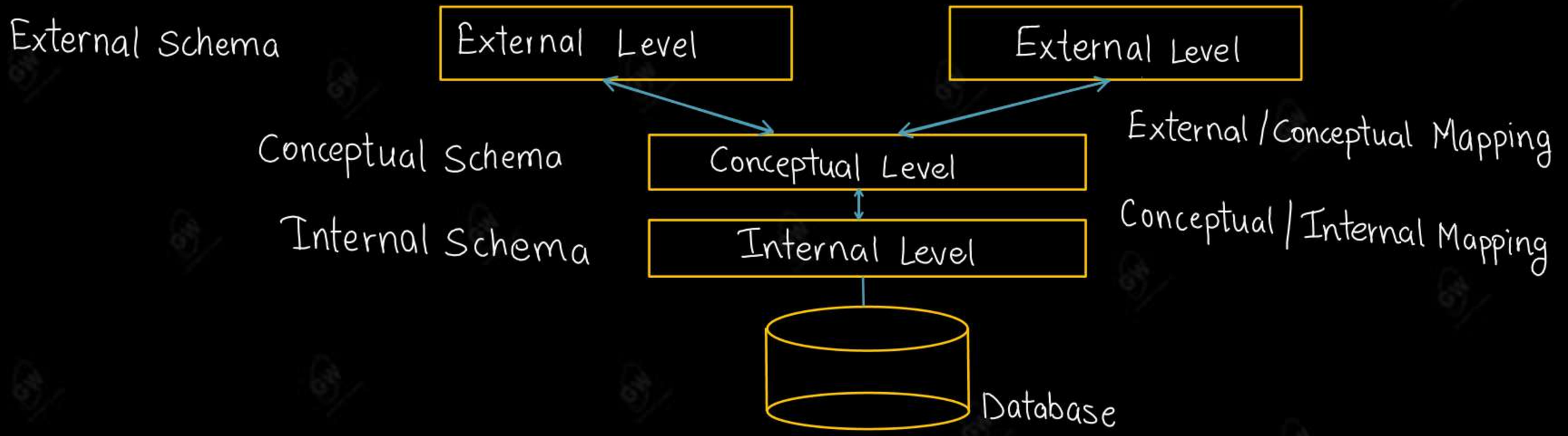
| Interface Type | Description | Used By | Example |
|---|---|---|---|
| 4. Natural Language Interface | Allows interaction using everyday language instead of SQL or commands. | Novice users | "Show all students with marks > 90" |
| 5. Command-line Interface (CLI) | Requires users to type commands or queries (like SQL). | Developers, DBAs | MySQL shell, Oracle SQL*Plus |
| 6. Application Program Interface (API) | Enables programs to communicate with the DBMS using programming languages. | Application developers | JDBC (Java), ODBC (C/C++), Python DB API |
| 7. Voice-based Interface | Uses speech recognition to query the database. | Voice assistant applications | “Find nearest hospital” via voice input |

# Three-Schema Architecture AKTU 2018-19/2022-23

It is a framework used in database management systems (DBMS) to separate the user applications from the physical database. It defines three levels of abstraction

| Schema Level | Meaning | Details | Who Uses It | Example |
|---|---|---|---|---|
| External Schema | Also called View Level, represents user-specific views of the database | Each user sees only the data they need., Different users can have different views.- Hides irrelevant data for security or simplicity. | End users, Application programs | A customer can see their name and orders, but not internal IDs or salaries. |
| Conceptual Schema | Also called Logical Level; shows overall logical structure of the database | Describes all entities, attributes, and relationships., Includes constraints and rules.- Independent of physical storage. | Database designers, DBA | Defines tables like Customer, Orders with fields and their relationships. |

| Schema Level | Meaning | Details | Who Uses It | Example |
|---|---|---|---|---|
| Internal Schema | Also called Physical Level; shows how data is physically stored in the system | - Describes storage methods (e.g., indexing, partitioning).- Focus on performance, storage efficiency.- Closest to hardware. | System developers, DBMS | Data stored in B-trees, indexed by customer ID, stored in disk blocks. |



# Mapping

In Three-Schema Architecture, mappings are used to connect the different schema levels and ensure smooth data translation between them.
There are two types of mappings:

| Mapping Type | Connects | Purpose | Explanation |
|---|---|---|---|
| 1. External/Conceptual Mapping | External Schema ↔ Conceptual Schema | Maps each user view to the overall logical structure | Ensures that user views (external schema) match the conceptual schema. If changes occur in the conceptual schema, the user view can remain unchanged. |
| 2. Conceptual/Internal Mapping | Conceptual Schema ↔ Internal Schema | Maps the logical design to the physical storage structure | Ensures that changes in how data is stored (physical level) do not affect the logical structure of the database (conceptual schema). |

# Database language AKTU 2017-18/18-19/20-21

In DBMS, Database Languages are used to create, modify, retrieve, and manage data in a database.
It is categorize into 4 parts
DDL,DCL,DML,TCL

## 1. DDL

DDL (Data Definition Language) is used to define and manage database schema, structure, and storage details.
It creates, modifies, and deletes database objects like tables, views, indexes.

| Command | Purpose | Example |
|---|---|---|
| CREATE | Creates a new table, view, schema, or index | CREATE TABLE Student (ID INT, Name VARCHAR(50)); |
| ALTER | Modifies an existing database object (add/remove columns, change type) | ALTER TABLE Student ADD Age INT; |
| DROP | Deletes database objects permanently | DROP TABLE Student; |
| RENAME | Renames a table or column | RENAME TABLE Student TO Learner; |
| TRUNCATE | Removes all records from a table but keeps its structure | TRUNCATE TABLE Student; |

# DML

- Data Manipulation Language (DML) is used to retrieve, insert, update, and delete data stored in the database.
  It works on data inside the schema defined by DDL.

| Command | Purpose | Example |
|---|---|---|
| SELECT | Retrieve data from one or more tables | SELECT * FROM Student; |
| INSERT | Add new data into a table | INSERT INTO Student VALUES (1, 'Amit', 'CS'); |
| UPDATE | Modify existing data | UPDATE Student SET Dept = 'IT' WHERE RollNo = 1; |
| DELETE | Remove records from a table | DELETE FROM Student WHERE RollNo = 1; |

# DCL

DCL (Data Control Language) is used to control access to data in a database.
It manages permissions and authorization for users.

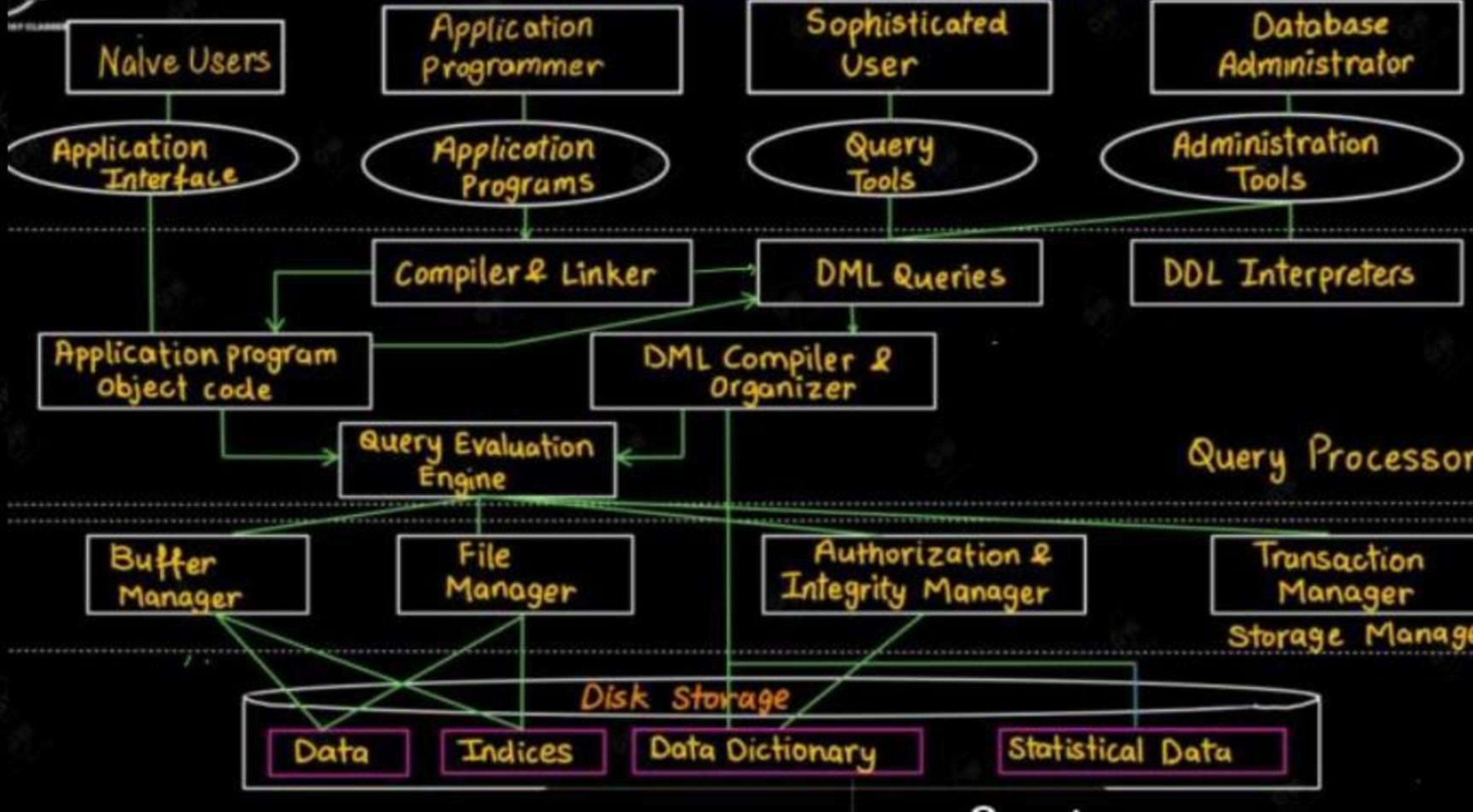
| Command | Purpose | Example |
|---|---|---|
| GRANT | Gives privileges to users (e.g., read, write, execute access) | GRANT SELECT, INSERT ON Student TO user1; |
| REVOKE | Removes previously granted privileges from users | REVOKE INSERT ON Student FROM user1; |

# TYPES OF DML 2022-23

| Aspect | Procedural DML | Non-Procedural DML |
|---|---|---|
| Meaning | Specifies how to retrieve or manipulate data step-by-step | Specifies what data to retrieve without detailing how |
| User Control | User specifies the sequence of operations | User specifies only the desired result |
| Complexity | More complex, requires understanding of data access paths | Simpler and easier to use |
| Examples | Relational Algebra, Relational Calculus | SQL (Structured Query Language) |
| Level of Abstraction | Low-level language, closer to physical data manipulation | High-level language, abstracts data access details |
| Who Uses It | Database programmers, system developers | End users, analysts, application developers |
| Execution | Requires explicit instructions on how to access data | DBMS optimizes query execution automatically |
| Flexibility | More control on data retrieval process | Less control but more user-friendly |

AKTU 2018-19/2023-24

# OVERALL ARCHITECTURE OF DBMS



## Application Programmers

- Write application programs using programming languages.
- Use RAD tools to build forms and reports without writing code

## Sophisticated Users

- Use query languages (like SQL) directly.
- Submit queries to the query processor without writing full programs.

## Specialized Users

- Develop complex database applications (e.g., CAD, expert systems, multimedia systems).
- Work outside the traditional database model.

.

## Naïve Users

Use pre-written application programs.

Example: A bank teller using a transfer program

## Database Administrator (DBA)

The DBA manages and coordinates all activities of the database system, ensuring data is secure, consistent, and available.

Some responsibility

- Creates the database structure using DDL commands.
- Defines how data is stored and accessed.
- Updates structure to match new needs or improve performance.

## Parts of Query processor:

### DDL interpreter

This will interprets DDL statements and fetch the definitions in the data dictionary.

### DML compiler

a. This will translates DML statements in a query language into low level instructions that the query evaluation engine understands.

b. A query can usually be translated into any of a number of alternative evaluation plans for same query result DML compiler will select best plan for query optimization.

### Query evaluation engine

This engine will execute low-level instructions generated by the DML compiler on DBMS

# Storage Manager/Storage Management:

A storage manager is a program module which acts like interface between the data stored in a database and the application programs and queries submitted to the system.

Thus, the storage manager is responsible for storing, retrieving and updating data in the database

**The storage manager components include:**

- **1.Authorization and integrity manager:**

Checks for integrity constraints and authority of users to access data.

- **2.Transaction manager:** Ensures that the database remains in a consistent state although there are system failures.
- **3.File manager:** Manages the allocation of space on disk storage and the data structures used to represent information stored on disk.
- **4.Buffer manager:** It is responsible for retrieving data from disk storage into main memory. It enables the database to handle data sizes that are much larger than the size of main memory.

## Data structures implemented by storage manager.

1. **Data files:** Stored in the database itself.
2. **Data dictionary:** Stores metadata about the structure of the database.
3. **Indices:** Provide fast access to data items.

**Q.1** In a university, a student enrolled in course and a student be assigned at least one or more course .Each course is taught by a single professor To maintain instruction quality a professor can delivery only one course **(AKTU 2015-16)**

① Entity Set

→ Student
→ Course
→ Professor

② Attribute

Student

Student_id , Student_name

Student_id (P.K.)

Address

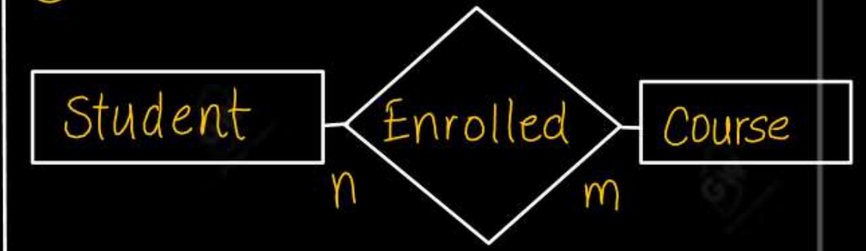
Course

Course_id, course_name, Duration Course_id (P.K)

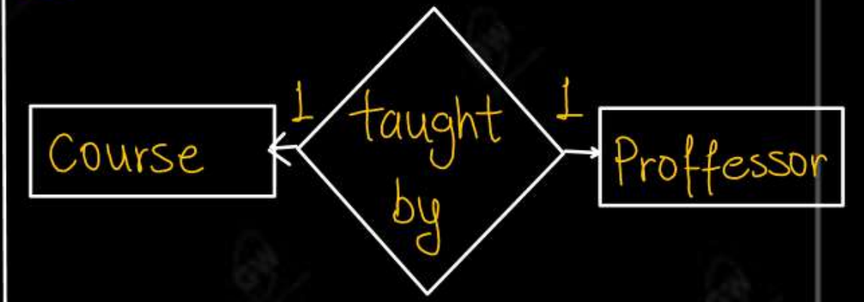
Professor

Professor_name, professor_id professor_id (P.K)

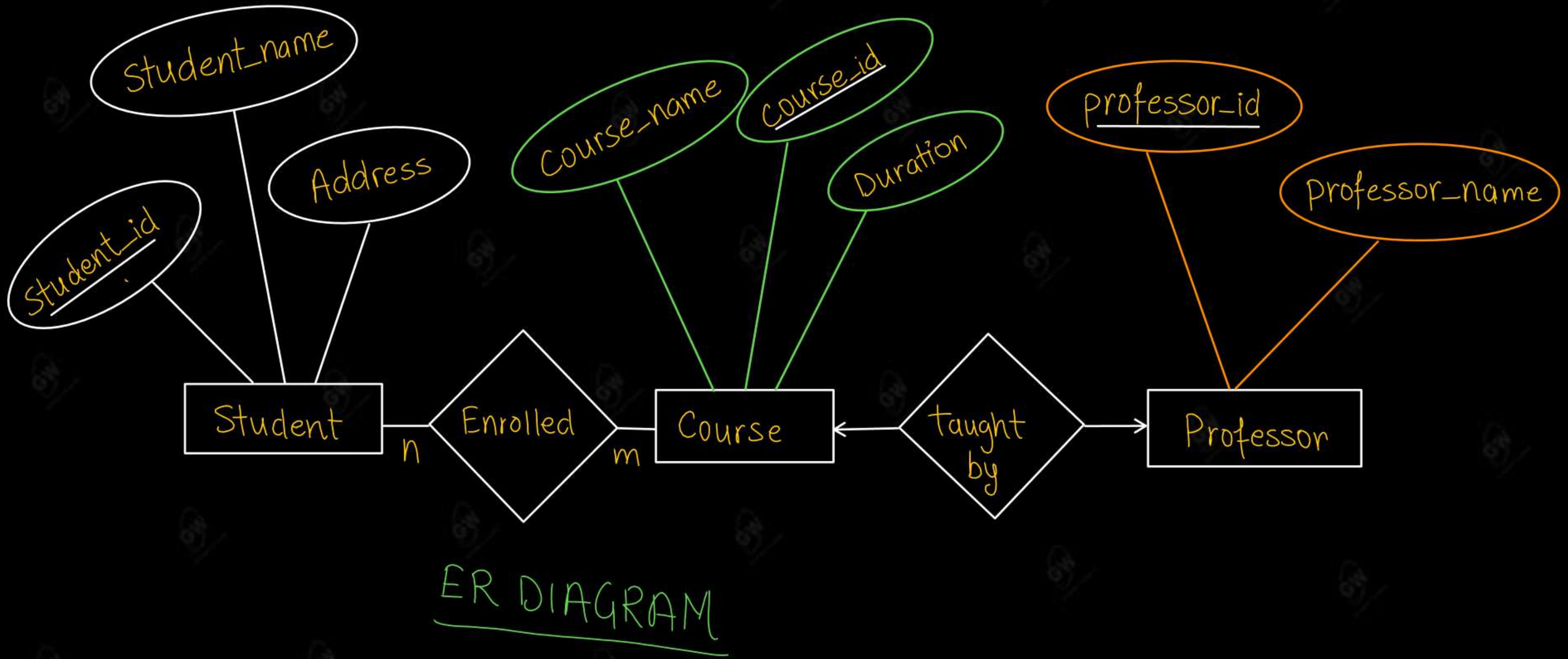
①




②




(Relationship Enrolled taught by


Student_name
Student_id
Address
Course_name
Course_id
Duration
professor_id
Professor_name
Student
n
Enrolled
m
Course
taught by
Professor

ER DIAGRAM

## Q.2 Draw an ER Model for an University database application where

a) A University has many departments.

b) Each department has multiple instructors; one among them is the head of the department

c) An instructor belongs to only one department.

d) Each department offers multiple courses, each of which is taught by a single instructor.

e) e) A student may enroll for many courses offered by different department (AKTU 2018-19)

| Entity set | |
|---|---|
| Department | Department<br>Department_id, Deptname, Location |
| Instructor | Instructor<br>Instname, Inst_id, Room no |
| Course | Course<br>Course_id, Coursename, duration |
| Student | Student<br>Student_id, Student_name, Address, DOB |

1.




2.

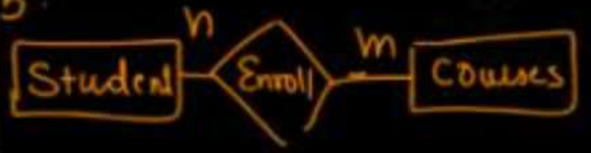



or

3.

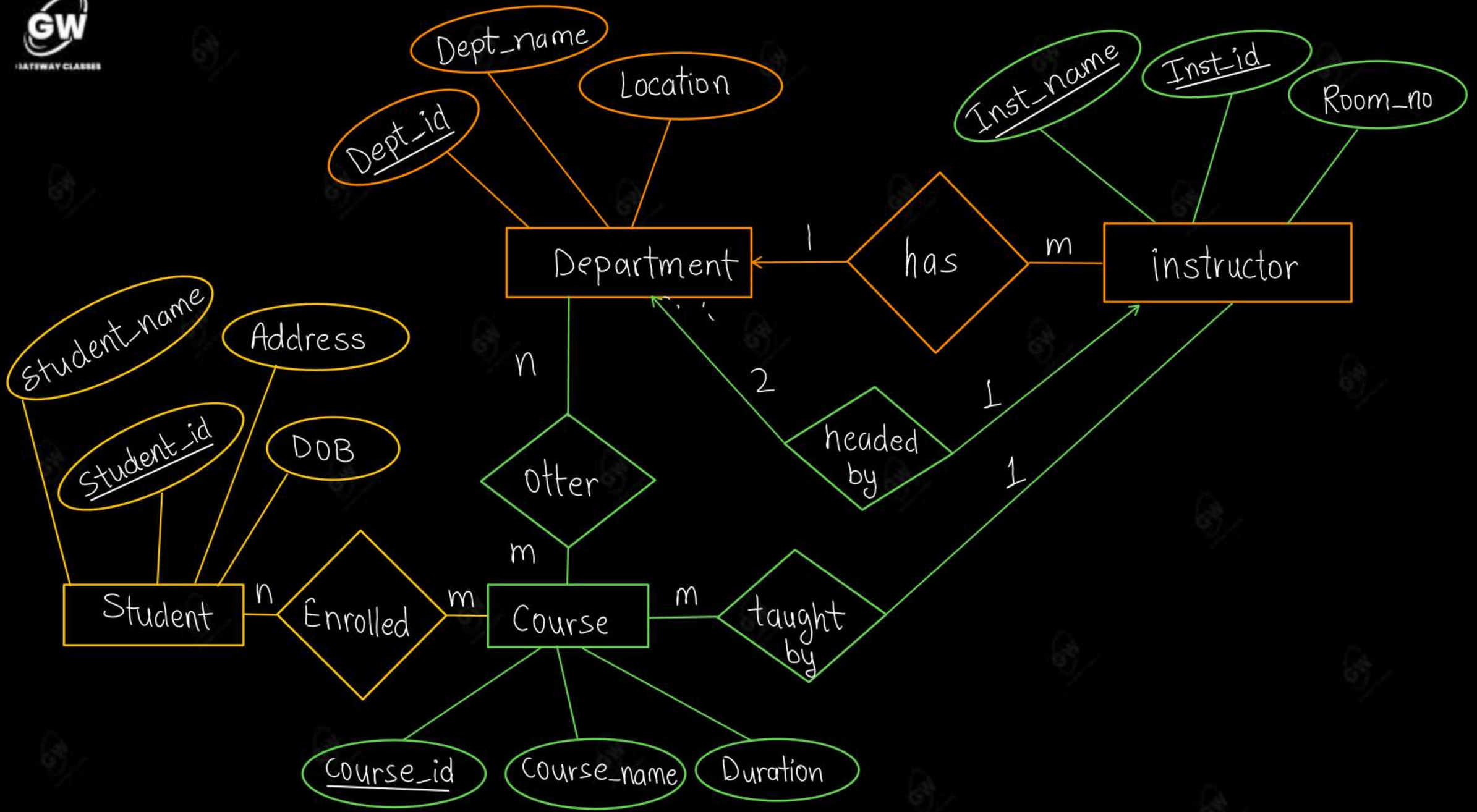



5.




4.




Dept_name
Location
Dept_id
Inst_name
Inst_id
Room_no
Department
1
has
m
instructor
n
2
1
Student_name
Address
Student_id
DOB
Otter
headed by
1
m
Student
n
Enrolled
m
Course
m
taught by
Course_id
Course_name
Duration

**Q.3** Construct an E-R diagram for a car insurance company whose customers own one or more cars each. Each car associated with it zero to any number

Entity set

Person
Car
Accident

Attribute

① Person

Person_id, Person_name
Address

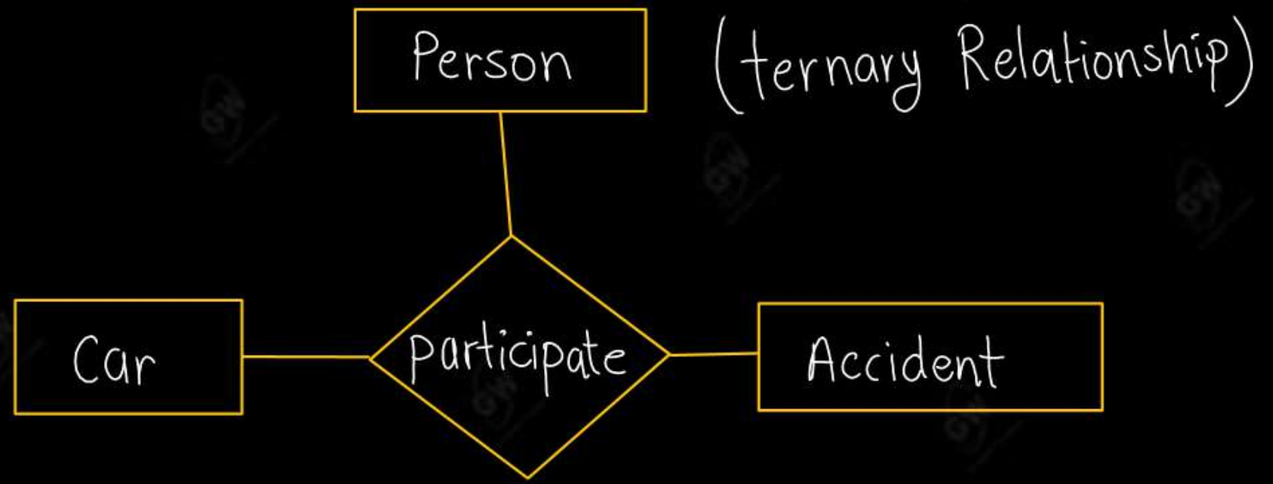
② Car

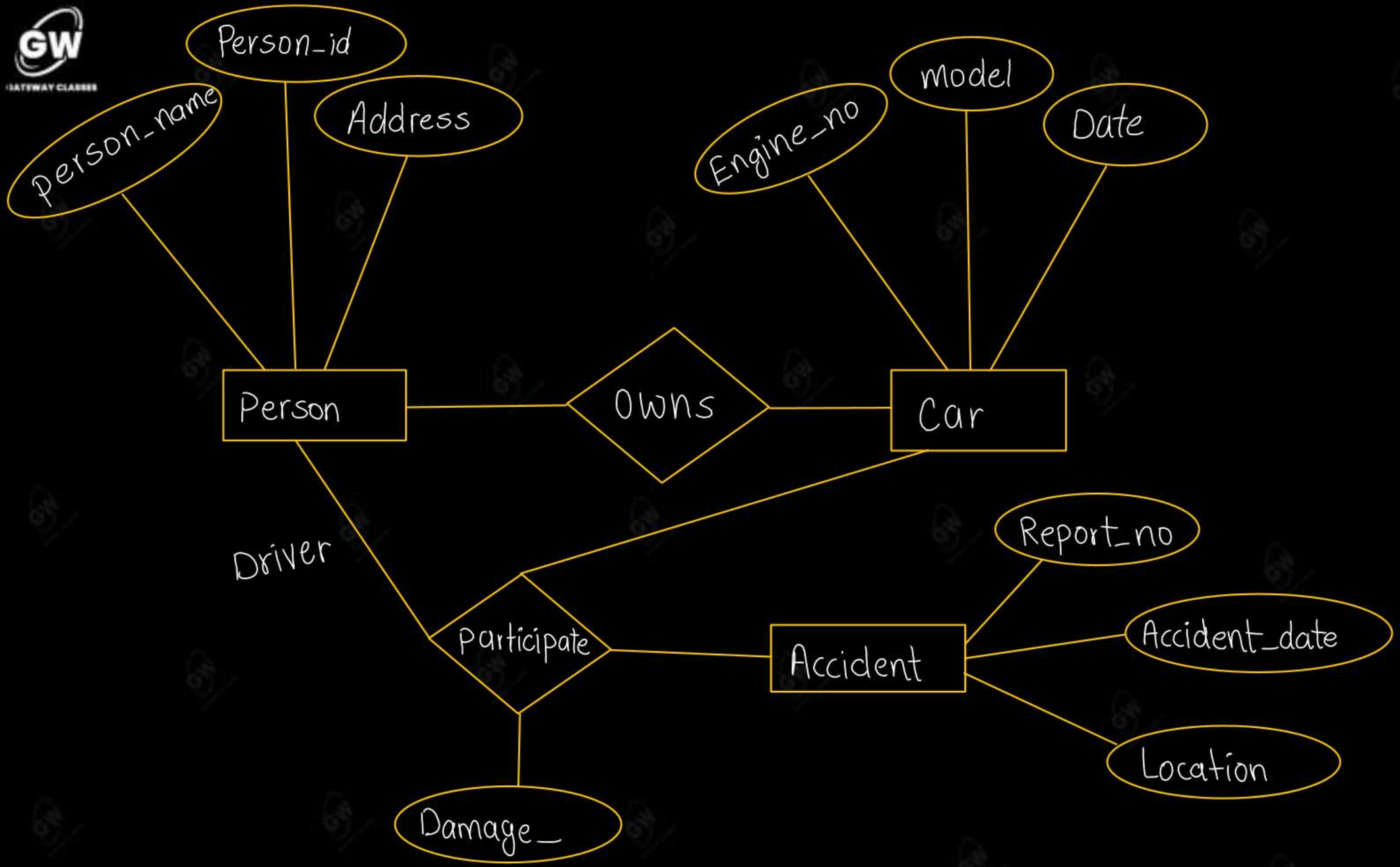
Engine_no, model, Date

③ Accident

Report_no
Accident_date
Location




Person_id
Person_name
Address
model
Engine_no
Date
Person
Owns
Car
Driver
Report_no
participate
Accident
Accident_date
Location
Damage_

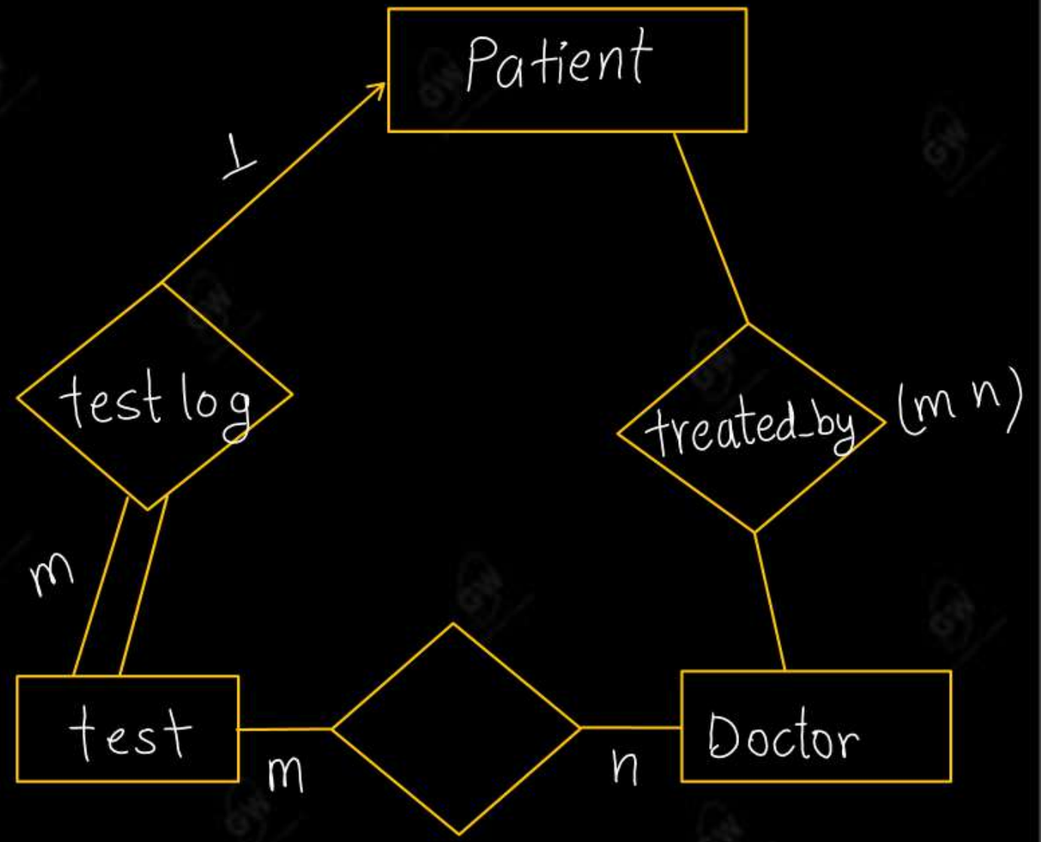
# Support of Different Definition Languages in Three-Schema Architecture

## AKTU 2019-20

| Schema Level | Definition Language | Description / Purpose |
|---|---|---|
| Internal Schema | Storage Definition Language (SDL) | Describes how data is physically stored, including file structures and indexing methods. |
| Conceptual Schema | Data Definition Language (DDL) | Defines the logical structure of the database (tables, constraints, relationships). |
| External Schema | View Definition Language (VDL) | Defines user-specific views of the data; manages access to subsets of the database. |

## Q.4 Construct an E-R diagram for a hospital with a set of patients and a set of medical doctors. Associate with each patient a log of the various tests and examinations conducted

| Entity Set | Attribute | Strong attribute |
|---|---|---|
| 1. Patient | Patient (strong entity set) | Doctor |
| 2. Doctor | P_id, insurance, | D_id |
| 3. Test | admitted_date | name |
| | checkout_date | specialisation |
| | name | |
| | test | |
| | t_id, Date, result, (strong entity set) | |
| | name, time | |



(table)

Relational Schema

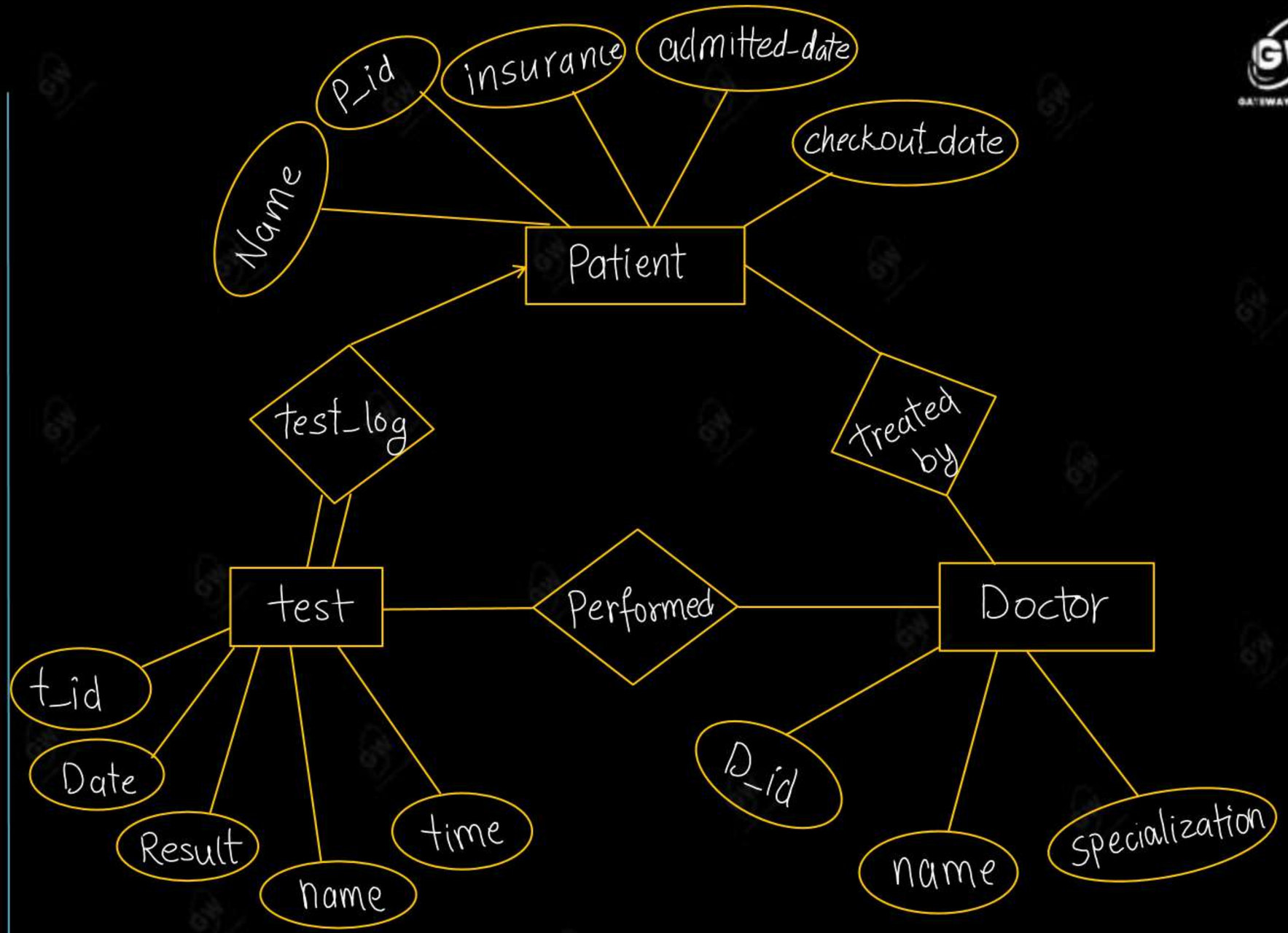
① Patient ( Name, P_id, insurance, admitted_date, check_out_date)

② Doctor(D_id, name, insurance)

③ treated by (D_id, P_id)

④ test (t_id, Date, Result, name time, P_id)

⑤ Performed_by(t_id, D_id)



**Q.5** An university registrar's office maintains data about the following entities:

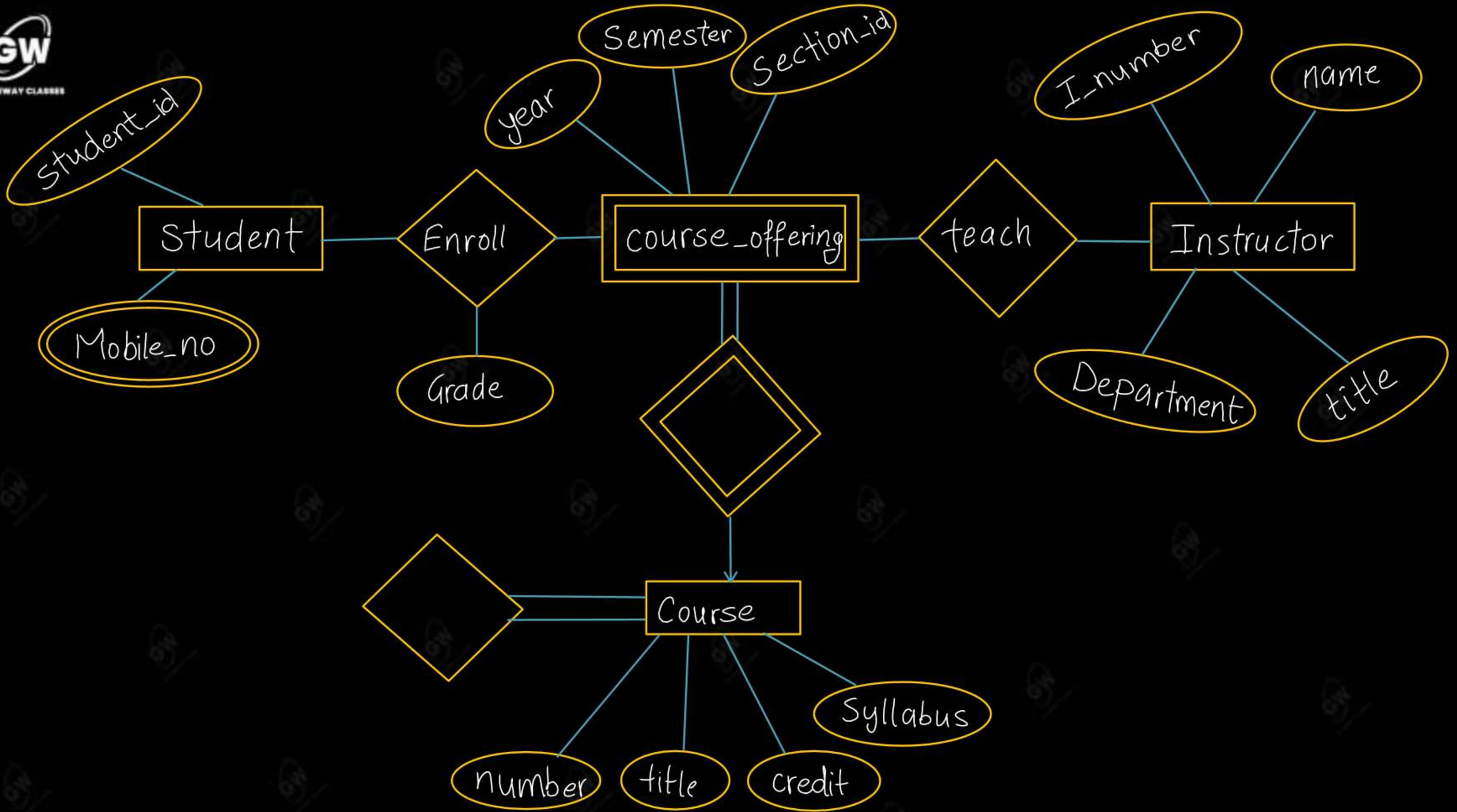
(a) courses, including number, title, credits, syllabus, and prerequisites;

(b) course offerings, including course number, year, semester, section number, instructor(s), timings, and classroom;

(c) students, including student-id, name, and program; and(

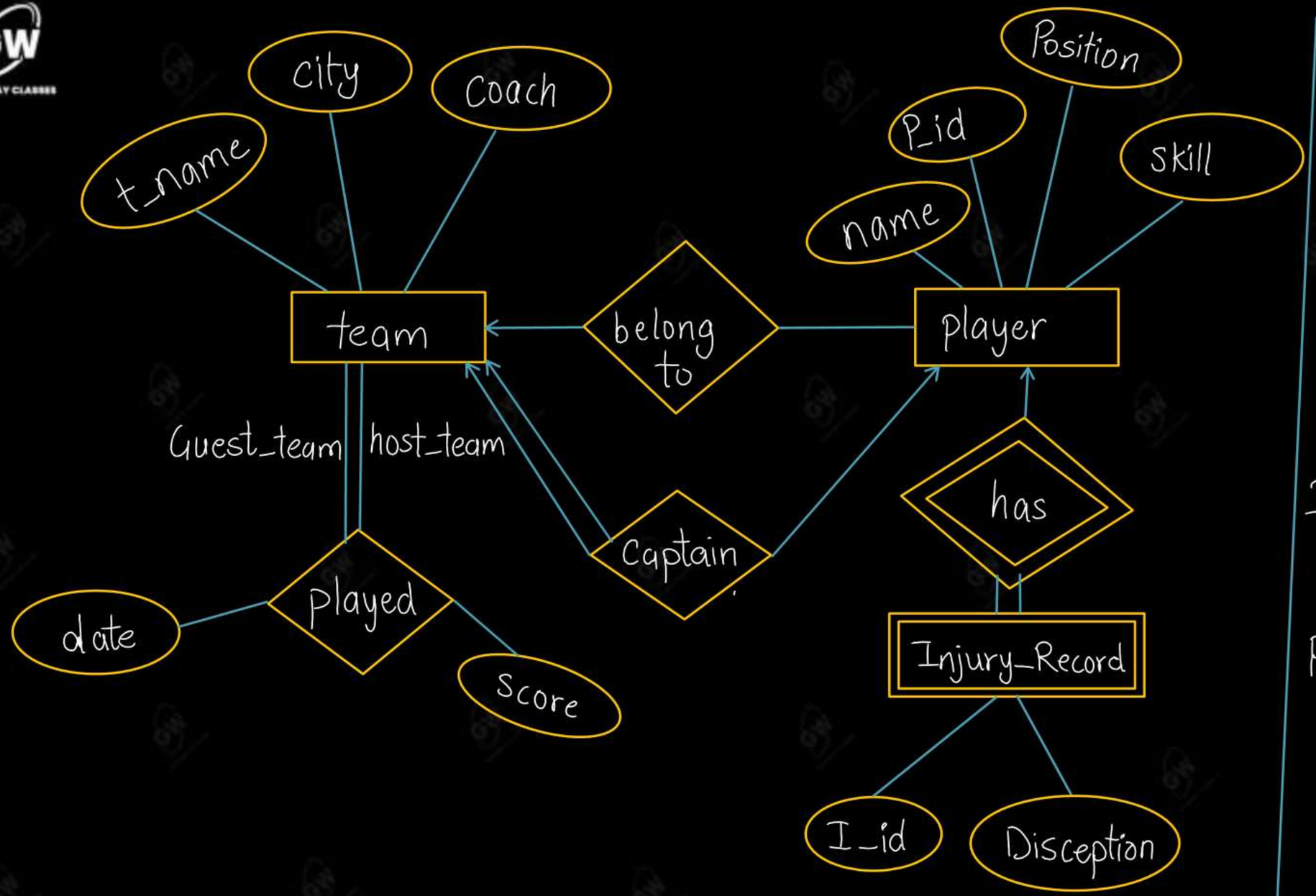
(d) instructors, including identification number, name, department, and title. Further, the enrollment of students in courses and grades awarded to students in each course they are enrolled for must be appropriately modeled. Construct an E-R diagram for the registrar's office. Document all assumptions that you make about the mapping constraints.

AKTU 2016-17


Semester
Section_id
year
I_number
name
Student_id
Student
Enroll
course_offering
teach
Instructor
Mobile_no
Grade
Department
title
Course
Syllabus
number
title
credit

**Q.6** Suppose you are given the following requirements for a simple database for the National Hockey League (NHL):

1. the NHL has many teams, each team has a name, a city, a coach, a captain, and a set of players, each player belongs to only one team,2 each player has a name, a position (such as left wing or goalie), a skill level, and a set of injury records,3 a team captain is also a player, 4. A game is played between two teams (referred to as host_team and guest_team) and has a date (such as May 11th, 1999) and a score (such as 4 to 2). Construct a clean and concise ER diagram for the NHL database



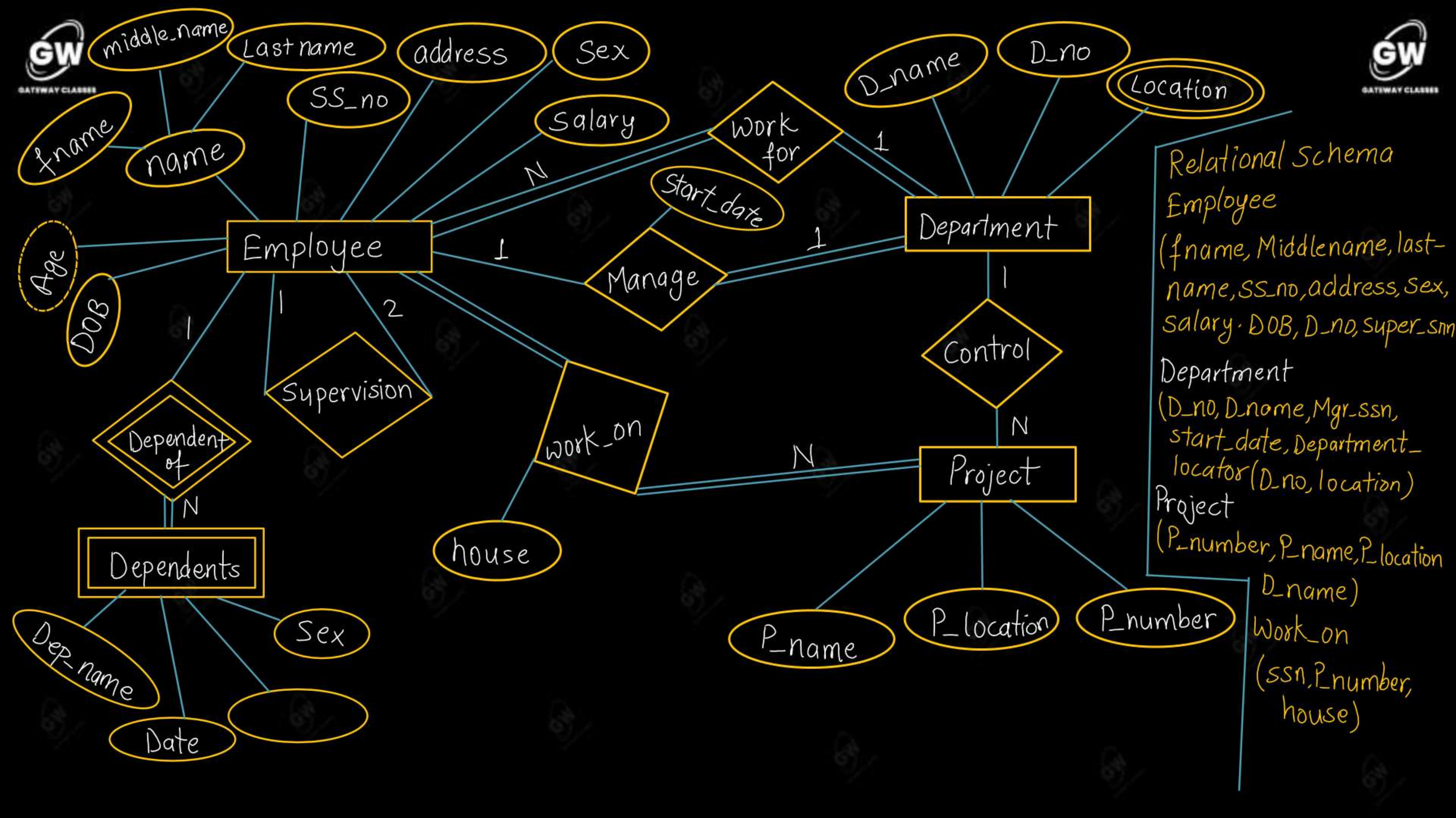

team(t_name, city, coach, captain_id)

Player(P_id, name, position, skill_level, t_name)

Injury_Record(P_id, I_id, Disception)

Played(Guest_team, host_team, t_name, date, score)

Q.7 Suppose you are given the following requirements for a simple database for GW the National Hockey League (NHL):only cap1. the NHL has many teams, each team has a name, a city, a coach, a captain, and a set of players, each player belongs to only one team,2 each player has a name, a position (such as left wing or goalie), a skill level, and a set of injury records,3 a team captain is also a player4. A game is played between two teams (referred to as host_team and guest_team) and has a date (such as May 11th, 1999) and a score (such as 4 to 2). Construct a clean and concise ER diagram for the NHL database.




Relational Schema

Employee
(fname, Middlename, last-name, SS_no, address, Sex, salary · DOB, D_no, super_ssn

Department
(D_no, D_nome, Mgr_ssn, start_date, Department_locator(D_no, location)

Project
(P_number, P_name, P_location D_name)

Work_on
(ssn, P_number, house)

## List some problems with file system AKTU 2022-23

| Problem | Explanation |
|---|---|
| Fragmentation | Files are stored in non-contiguous blocks, leading to slow read/write speeds. |
| Data Loss/Corruption | Sudden shutdown or crashes may cause file damage or loss of data. |
| Limited File Name Length | Some file systems restrict file name length or character types. |
| Inadequate Security | Basic file systems lack user access control or encryption features. |
| Storage Limitations | Older systems (like FAT32) have limits on file size and partition capacity. |
| Concurrency Issues | Multiple users/processes accessing files may cause conflicts or inconsistency. |
| Poor Large File Support | Inefficient handling of large files leads to slow performance or failure. |
| Lack of Journaling | Without journaling, system can't recover well from unexpected failures. |

## What are the four advantages of file system over database
## AKTU 2020-21/ 2019-20/ 2018-19

| Advantage | Explanation |
|---|---|
| Simplicity | File systems are easy to design and use for small applications. |
| Less Overhead | No need for complex software or hardware; runs directly on the OS. |
| Faster for Small Tasks | For simple storage/retrieval, file systems may be quicker than DBMS. |
| Low Cost | No need to purchase or maintain expensive database management software. |

## Database instance AKTU 2017-18/2022-23

- A **database instance** is the **current state of the data** stored in a database at a specific moment in time.
  It is like a **snapshot** of all the data in the database at that moment.

# Metadata in DBMS AKTU 2022-23

Metadata in DBMS is data that describes the structure and properties of database data.

| Metadata Type | Example |
|---|---|
| Table Name | Students, Employees |
| Column Names | RollNo, Name, Age, Salary |
| Data Types | INT, VARCHAR, DATE, etc. |
| Constraints | PRIMARY KEY, NOT NULL, FOREIGN KEY |
| Relationships | Table Orders linked to table Customers |
| Indexes | Index on RollNo for fast searching |
| Views | Virtual tables created from one or more real tables |
| Metadata Type | Example |

## EER Diagram

An Enhanced ER (EER) diagram is an extension of the basic Entity-Relationship (ER) diagram. It includes all features of ER diagrams along with additional concepts that support more complex database designs such as specialization, generalization, and categorization.

In Enhanced ER Diagram visually represents entities, relationships, and additional concepts like:

- Subclasses and Superclasses
- Specialization / Generalization
- Aggregation
- Categorization (Union Types)

## Super class and subclass

superclass is a high-level entity that can be further segmented into subclasses or subsets,

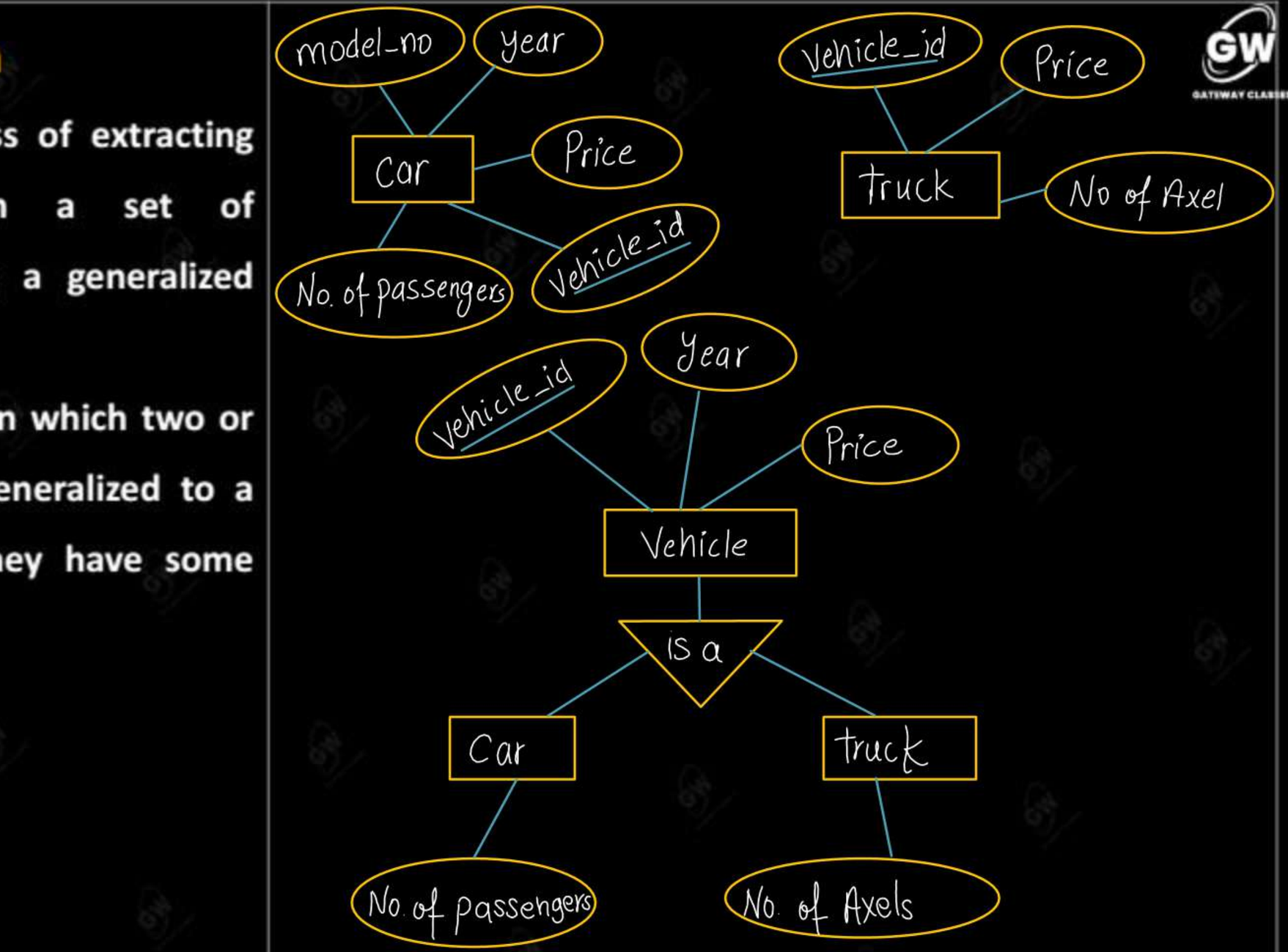
It is also referred to as a Parent class.

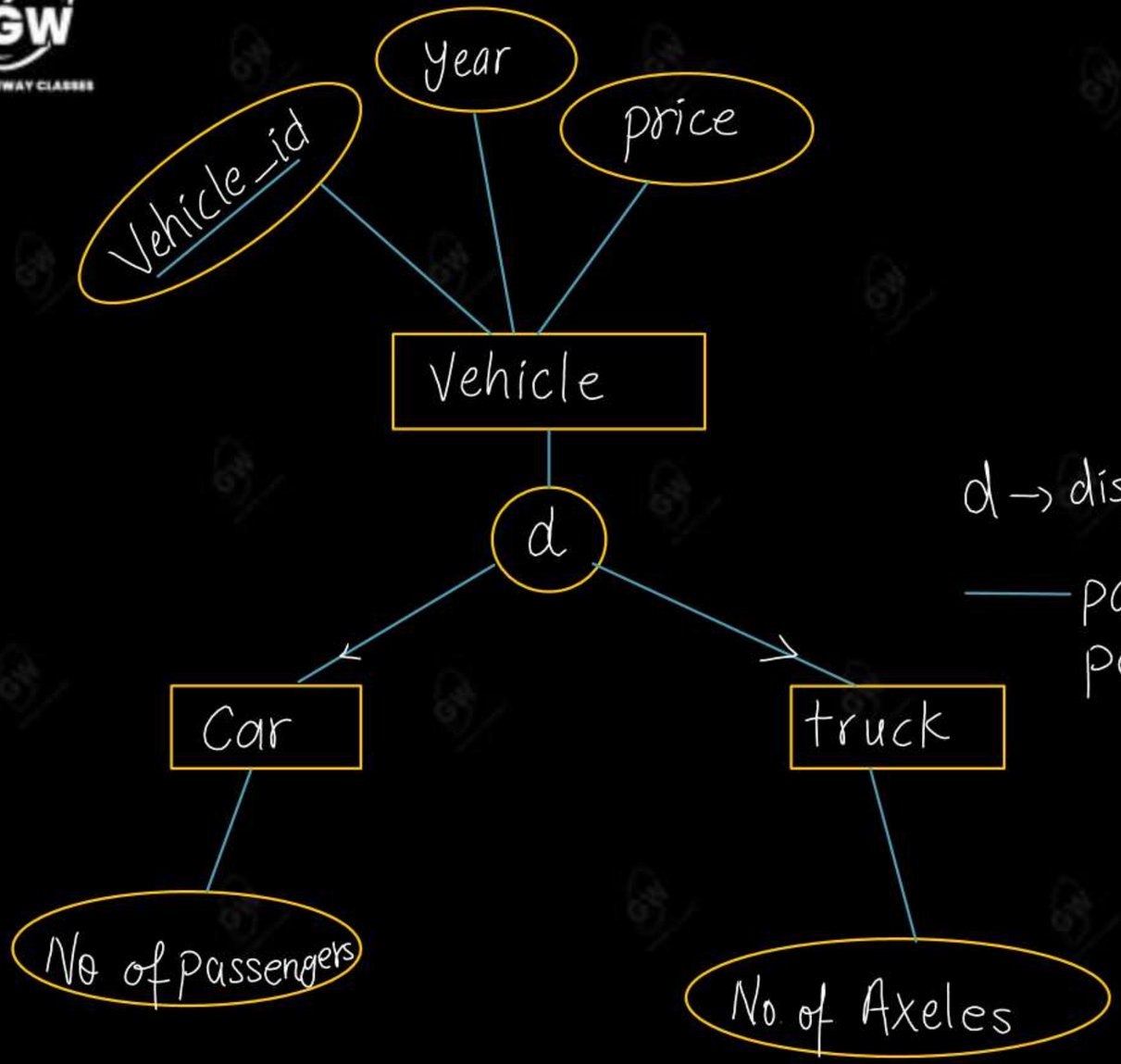
A subclass can be referred to as a child or derived class.

Example: Science is a Super class which has subclasses like Physics, Chemistry, Biology

# Generalization

- Generalization is the process of extracting common properties from a set of entities/entity and creating a generalized entity set from it.
- It is a bottom-up approach in which two or more entities set can be generalized to a higher-level entity set if they have some attributes in common.






d → disjoint

— partial participation

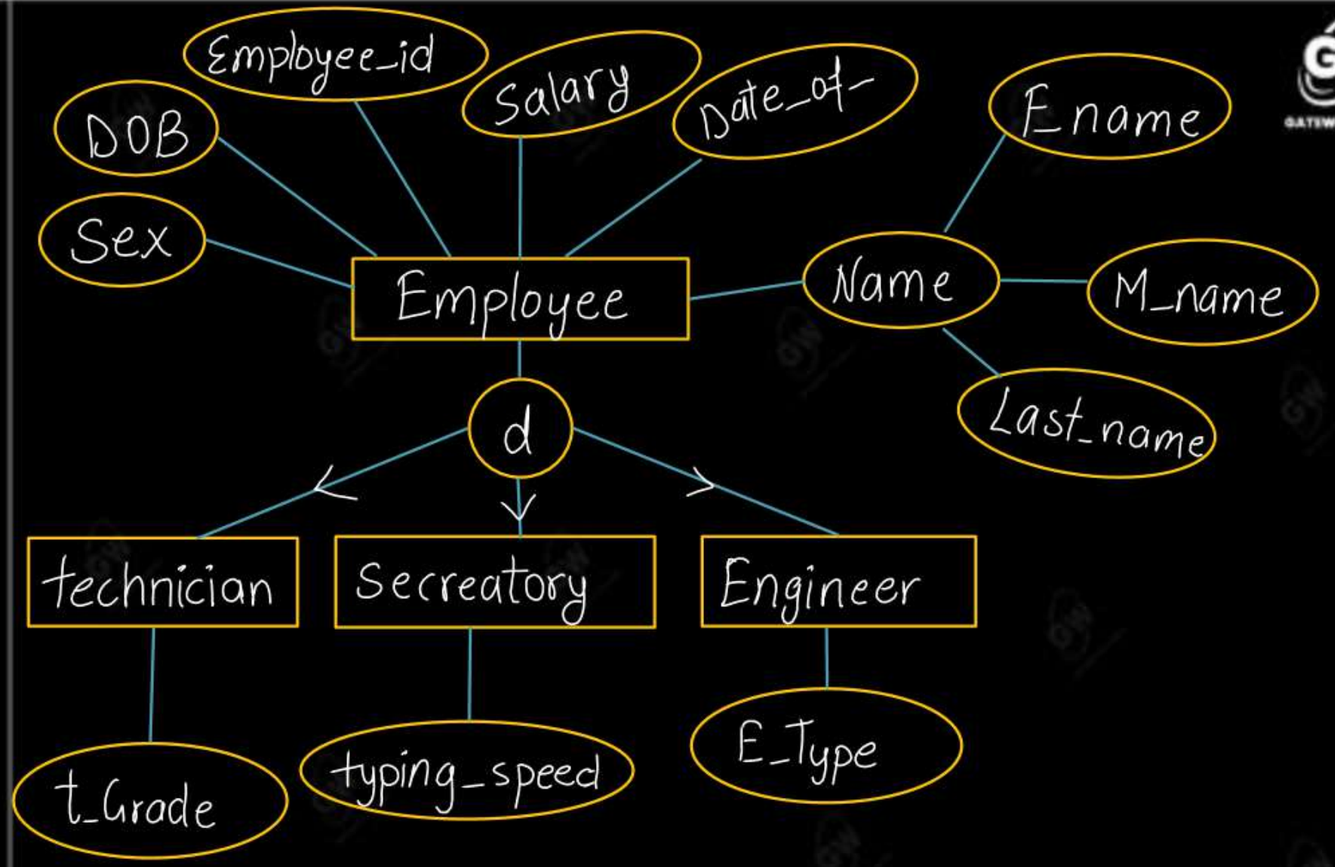
Generalisation
Generalization
(Bottom up approach)

Specialisation
(top-down approach)

# Specialization

specialization, an entity set is divided into sub-entities set based on its characteristics.

It is a top-down approach where the higher-level entity set is specialized into two or more lower-level_entities set




d → disjoint

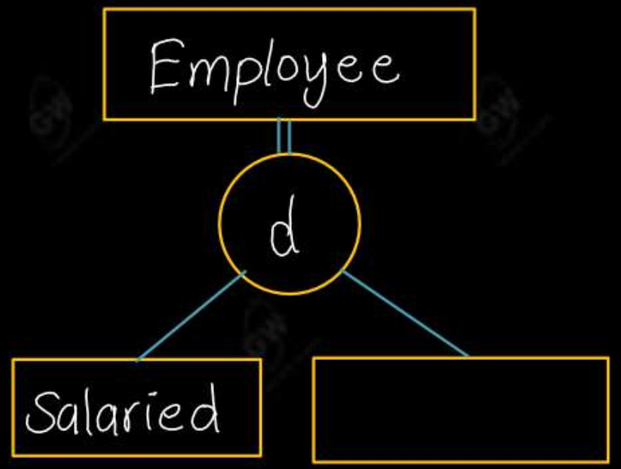
(Employee either a technician, Secreatory, employee)

Partial partial __

Attribute Constraints

technician → tgrade

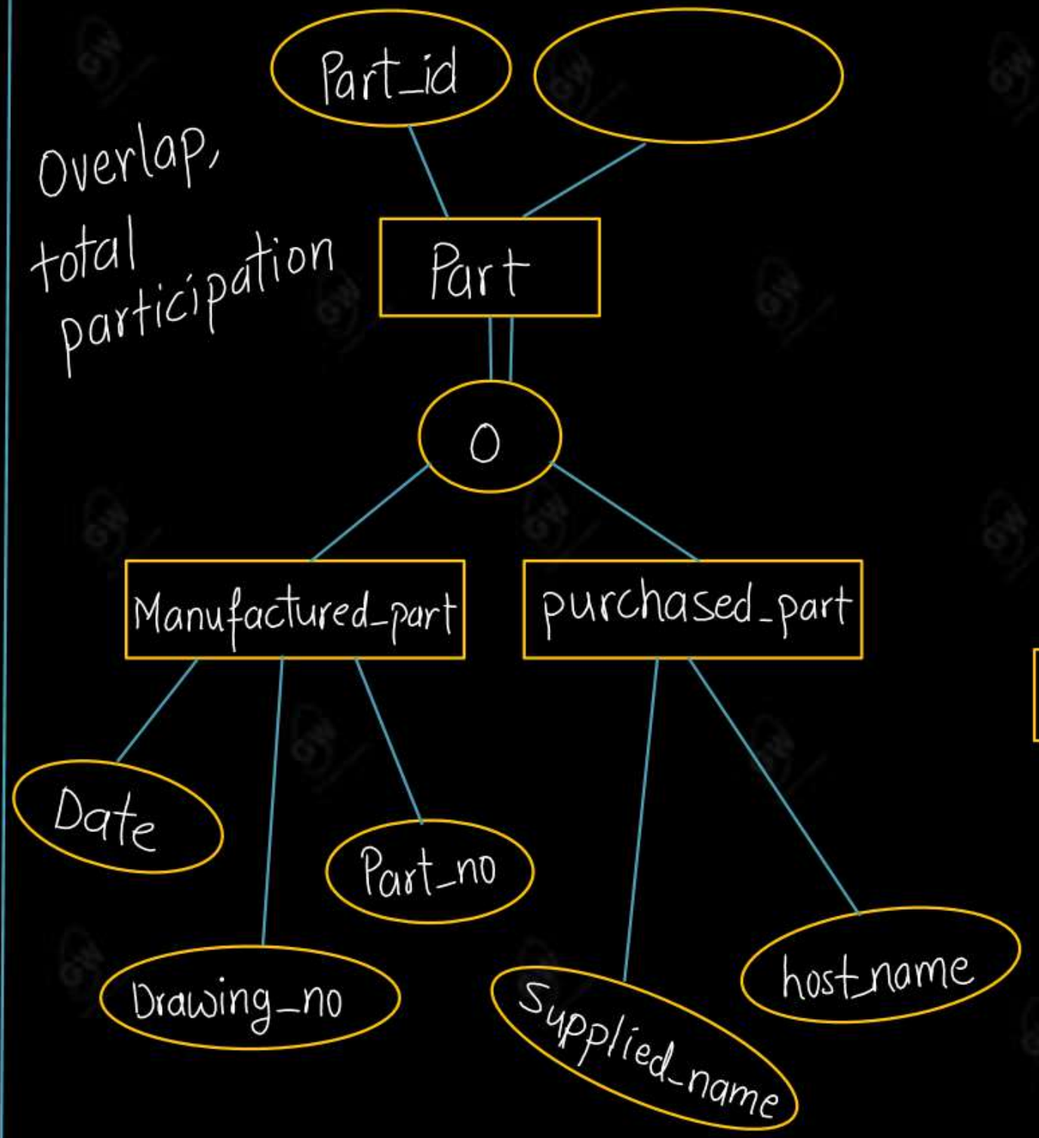
Sex, DOB, Employee_id, etc



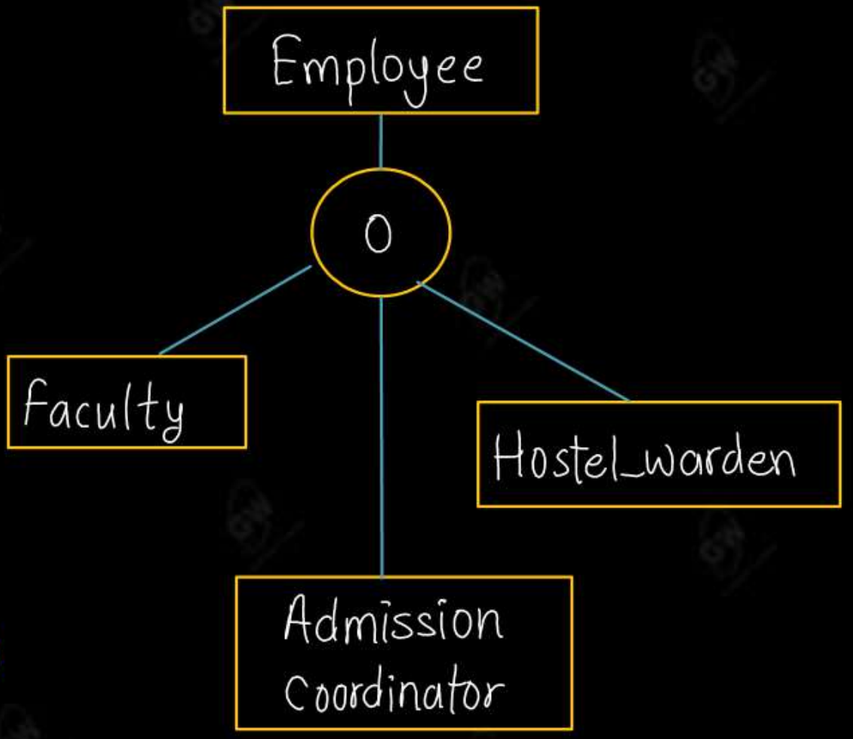

disjoint, total participation

Every employee either be Salaried or

Overlap, total participation




Overlap partial parti ipation



# Inheritance in DBMS (EER Model)?

Inheritance is a feature in Enhanced ER (EER) models where a subclass inherits attributes and relationships from a superclass.

**Example:**

If Person is a superclass, and Student is a subclass, then Student inherits Name, Age, etc., from Person.

## What is Attribute Inheritance?

In attribute inheritance, a subclass automatically inherits all attributes of its superclass.

**Example:**

Superclass: Vehicle (attributes: vehicle_no, color)

Subclass: Car → Inherits vehicle_no, color from Vehicle

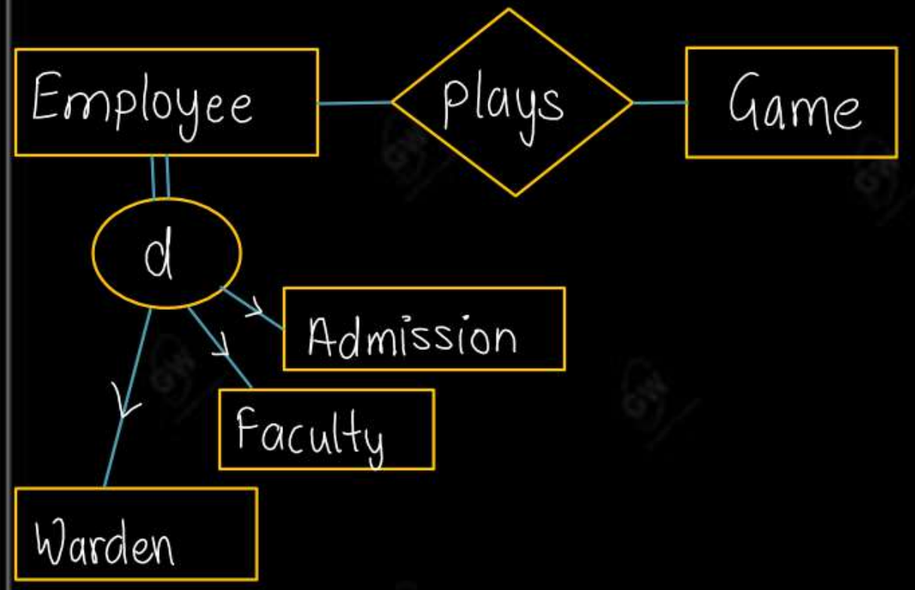
## What is Participation Inheritance?

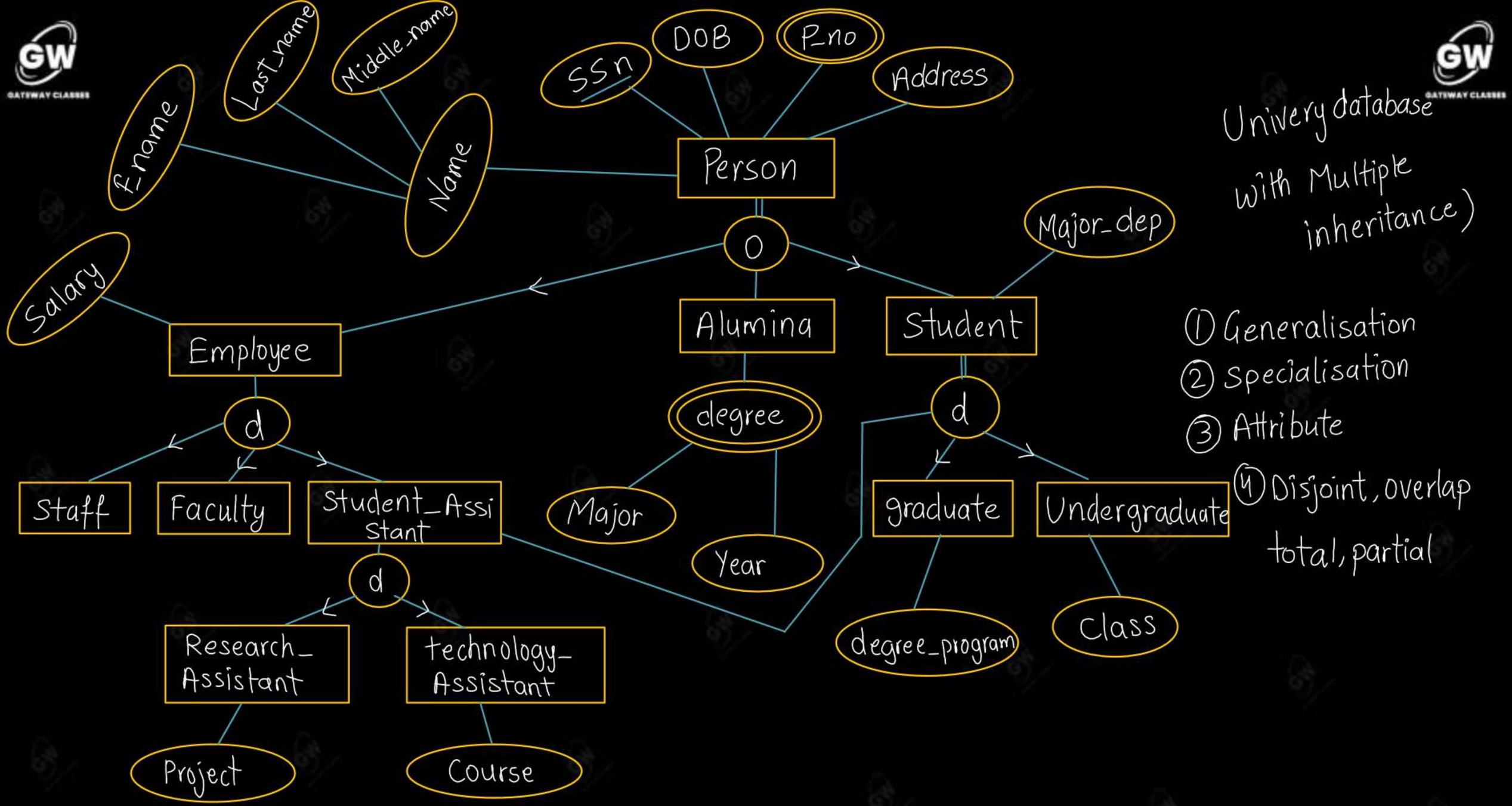
In participation inheritance, a subclass also inherits relationships of the superclass if it participates in them.

**Example:**

Employee works in Department

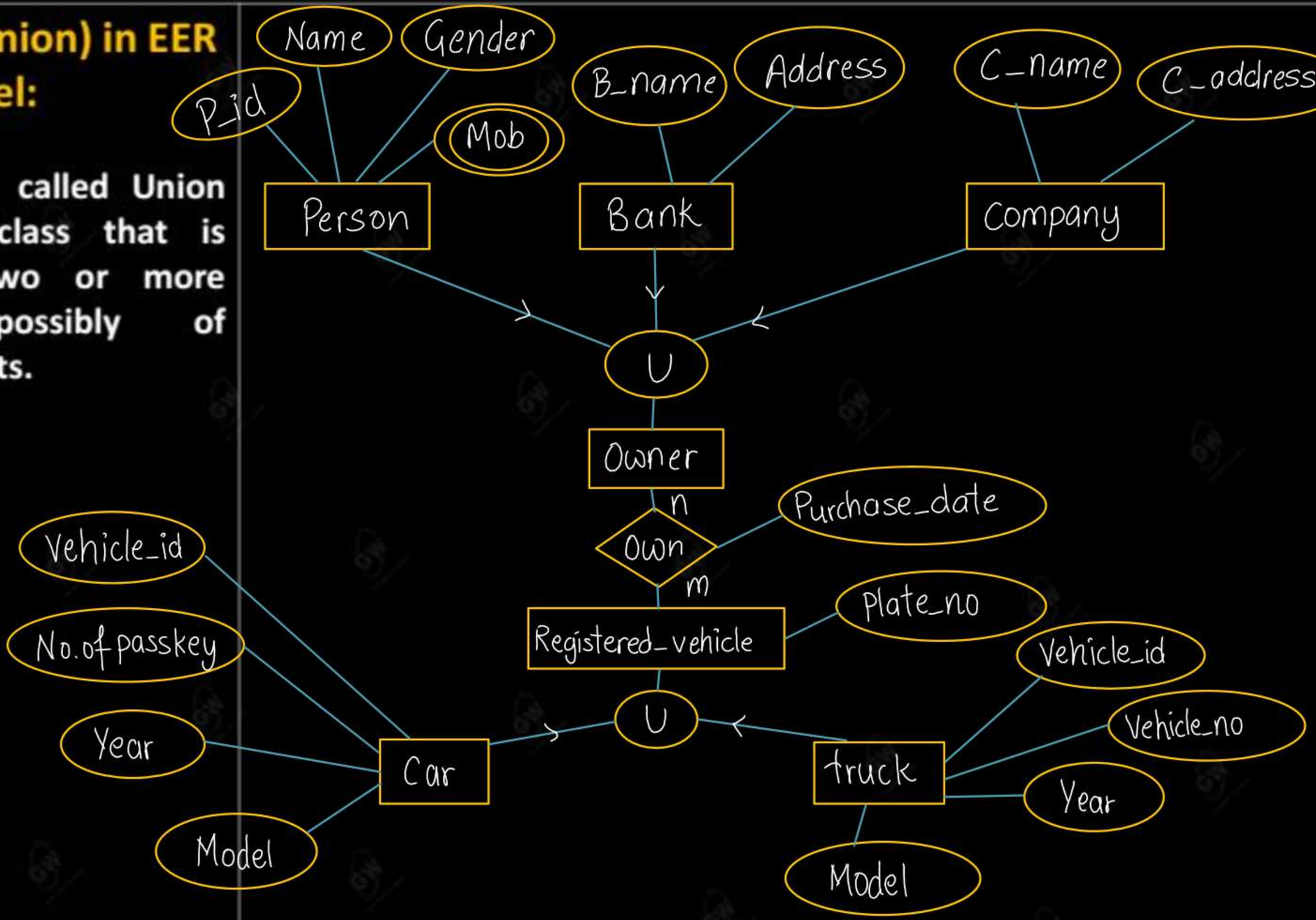
Subclass Manager (of Employee) → also participates in works in relationship




GW
GATEWAY CLASSES
F_name
Last_name
Middle_name
Name
SSn
DOB
P_no
Address
Person
O
Major_dep
Salary
Employee
Alumina
Student
d
degree
d
Staff
Faculty
Student_Assi Stant
Major
Year
graduate
Undergraduate
d
Research_ Assistant
technology_ Assistant
degree_program
Class
Project
Course
Univery database with Multiple inheritance)
① Generalisation
② Specialisation
③ Attribute
④ Disjoint, overlap
total, partial

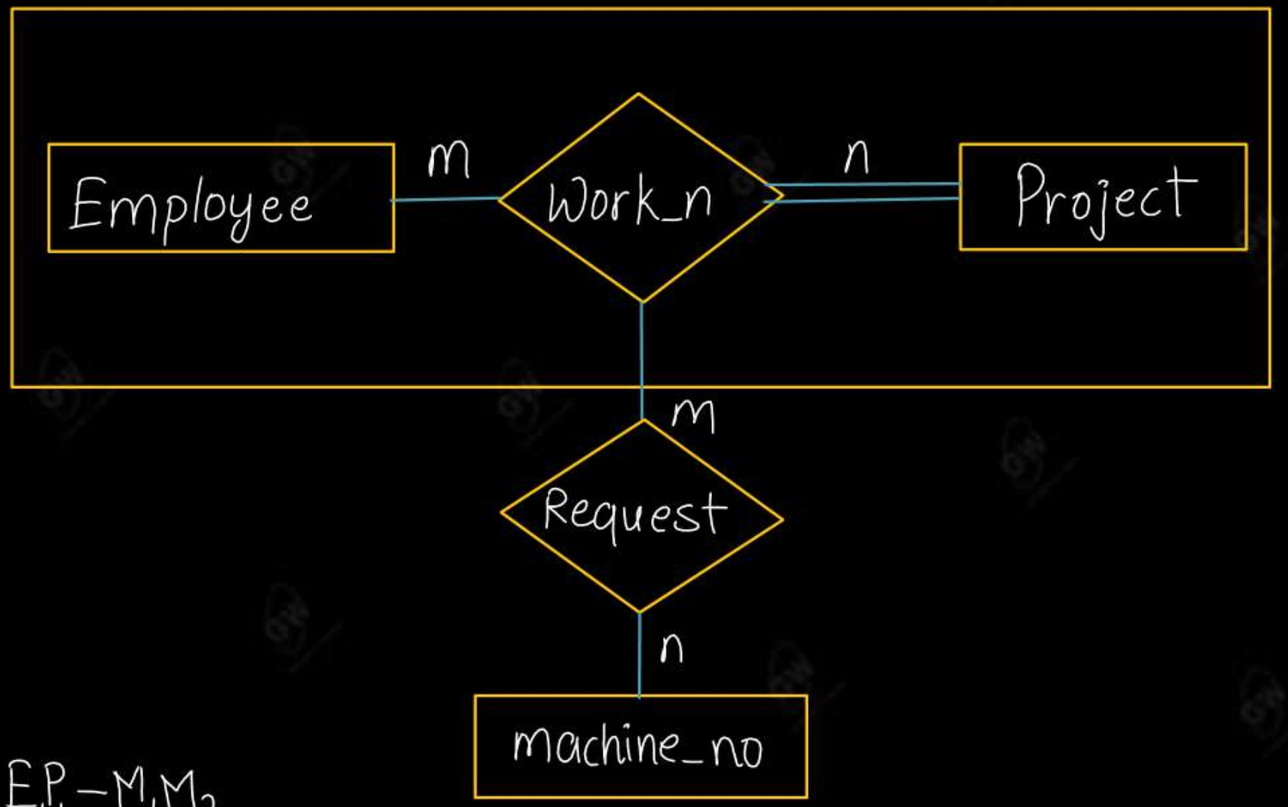
# Category (Union) in EER Model:

A Category (also called Union type) is a subclass that is derived from two or more superclasses, possibly of different entity sets.



# What is Aggregation in DBMS (EER Model)?

Aggregation is a concept used when a relationship itself needs to be treated as an entity and involved in another relationship.




$E_1P_1 - M_1M_2$

$E_1P_2 - M_1$

| AKTU PYQs | | |
|---|---|---|
| Q.1 | What are the different types of models in DBMS? Explain them | AKTU 2022-23 /2020-21/2018-19/ 2017-18 |
| Q.2 | Explain different types of keys<br>OR Difference between primary key and unique key.<br>OR Explain super key, Primary key, candidate key. | AKTU 2020-21/2017-18/ 2021-22 |
| Q.3 | What is the significance of physical data independence | AKTU 2020-21 |
| Q.4 | List four function of database administrator(DBA) | AKTU2022-21 |
| Q.5 | Explain DML, DDL | AKTU 2017-18/18-19/20-21 |
| Q.6 | What is instance and schema of database | AKTU 2017-18/2022-23 |
| Q.7 | Difference between physical and logical independence | AKTU 2022-23 |
| Q.8 | What is data abstraction? Explain the level of data abstraction<br>OR How data abstraction is achieve in DBMS? | AKTU 2022-23/ 2020-21 |

| ...AKTU PYQs | | |
|---|---|---|
| Q.9 | Briefly discuss about the difference between data system and file system | AKTU 2020-21/ 2019-20 |
| Q.10 | What is data independence and its types | AKTU 2020-21/ 2021-22 |
| Q.11. | Explain the difference between physical level, conceptual level and view level | AKTU 2017-18 |
| Q.12 | What is the role of database administrator | AKTU 2017-18 |
| Q.13 | What is database language | AKTU 2018-19 |
| Q.14 | Distinguish between strong entity set and weak entity set.ER diagram to illustrate the weak entity set | AKTU 2022-23/ 2019-20 |
| Q.15 | Explain ER diagram in detail. | AKTU 2018-19 |
| Q.16 | What is the key of a relation? Explain the difference between primary key, super key and candidate key. | AKTU 2015-16 |

| ...AKTU PYQs | | |
|---|---|---|
| Q.17 | Give example of a simple, composite, attribute of an entity | AKTU 2015-16 |
| Q.18 | Difference between DBMS and RDBMS | AKTU 2018-19 |
| Q.19 | Explain briefly the overall structure of DBMS and also explain its components | AKTU 2018-19 |
| Q.20 | Describe the three schema architecture why we need mapping in schema level | AKTU 2022-23 |
| Q.21 | Explain all user of database in detail | AKTU 2021-22 |
| Q.22 | List some problem with file system | AKTU 2022-23 |
| Q.23 | What is DBMS ,data and meta data | AKTU 2022-23 |
| Q.24 | State the procedural DML and non-procedural DML | AKTU 2020-21 |
| Q.25 | What are the four advantages of file system over database | AKTU 2022-23 |